Building the Code Agent

From Code Foundation Models, Long Context, and Trajectory Learning to Executable Software Engineering Agents

Jiguo Li[1] jiguolee@gmail.com

Research and Engineering Blueprint (Sources verified through 2026-08-07)

Abstract A Code Agent is not a ReAct loop appended to a code language model, but a system jointly composed of a code foundation model, repository-level representations, long context, tool protocols, executable environments, trajectory learning, and verifiable evaluation. This report presents a path to engineering implementation: during pre-training, high-quality source code, code-related text, and development history are used to establish priors over syntax, semantics, and modifications; during mid-training, repo-level/FIM, dependency-aware organization, real Pull Requests, and a long-context curriculum transfer capabilities from single-file generation to cross-file localization and modification; during post-training, executable tasks, successful and failed trajectories, SFT/preference learning/reinforcement learning, and verifiers form a closed loop; at runtime, a controlled Agent-Computer Interface, retrieval, editing, testing, and sandboxes convert model capabilities into reliable actions. The report further proposes a data schema, a hierarchical evaluation matrix, contamination governance, equal-token ablation designs, and stage-specific acceptance thresholds. The central conclusion is that effective context, environment reproducibility, and trajectory diversity determine the ceiling of Code Agents more than simply expanding the context window or accumulating successful trajectories; foundation-model capabilities, scaffolds, and test-time compute must be measured separately, or attribution is impossible.

Keywords Code Agent; code pre-training; repo-level; mid-training; long context; trajectory data; reinforcement learning; SWE-bench

Executive Summary: An implementable Code Agent should be built around three closed loops: the **data loop** converts code, PR/Issue/Commit data, and execution results into trainable signals; the **training loop** progressively transitions from next-token/FIM to tool use, long-horizon decision-making, and verifiable optimization; the **evaluation loop** stratifies function-level, cross-file, repository-level, and end-to-end tasks, while jointly constraining resolve rate through freshness, environment success rate, cost, and side effects. An increase in any single metric should not be overinterpreted as complete Agent capability.

[1] This report was prepared with assistance from Codex.

Contents

Notation and Conventions

This page collects symbols reused throughout the report; quantities local to one derivation are still defined at first use. Subscript t usually denotes an episode step, i, j index samples, positions, or tools, and θ, ϕ denote learnable parameters.

Symbol	Meaning	Main sections
$\mathcal{T}_j = (n_j, d_j, \Sigma_j^{\text{in}}, P_j, \Delta_j, \Sigma_j^{\text{out}})$	Contract of tool j: name, description, input schema, permission/scope, execution budget, and output schema.	1.1
$s_t, o_t, a_t, h_t, \pi_\theta, \tau$	State, observation, action, history, policy parameterized by θ, and a complete interaction trajectory (episode).	1.1–1.2
$M, S, H, E, V, B, \xi; Y$	Model checkpoint, scaffold/prompt, Harness, environment, verifier, budget, randomness; Y is the end-to-end outcome.	1.4, 5
$x_i, x_{i,<t}, m_{i,t}, p_\theta, w_d, q_i, T$	Training sample and prefix, loss mask, conditional token probability, domain weight, sample-quality score, and sampling temperature.	2
$\tilde{x}, x_{<l}, x_{l:r}, x_{>r}; L, n_{\text{layer}}, d_{\text{kv}}$	FIM reordering and the removed span; sequence length, layer count, and per-layer KV dimension for long-context cost estimates.	2.3–3
$m, i, b, d, \theta_i, \phi_i(m)$	RoPE position, frequency-dimension index, base, hidden dimension, frequency of dimension i, and positional phase.	3.2
$v(\tau), r_{\text{exec}}, r_{\text{progress}}$ $d_{\text{novelty}}, c_{\text{tokens}}, r_{\text{risk}}$	Trajectory value and its executability, progress, novelty, token-cost, and risk components; $\alpha, \beta, \gamma, \delta, \eta$ are mixture weights.	4.2
$\mathbf{r}(\tau), R_{\text{task}}, C, R_{\text{risk}}$ $s_\phi(\tau, P), \widehat{\text{pass@}k}$	RL reward vector, task return, cost, risk penalty, verifier score for a trajectory and patch, and pass@k estimated from c successes among n samples.	6–7

Overloaded Symbols and Abbreviations: d_j is a tool description, while d may also denote a data domain or hidden dimension; P_j is a tool permission, P a candidate patch, and P_ϕ a probability; T is sampling temperature, whereas $\mathcal{T}$ denotes a tool. Subscripts and context disambiguate them. Frequent abbreviations are NTP (next-token prediction), FIM (fill-in-the-middle), SFT, DPO, RL/RLVR, ACI (Agent–Computer Interface), F2P (fail-to-pass), and P2P (pass-to-pass).

1 Problem Definition: From Tool Use to the Code Agent Capability Stack

1.1 Tool Use: Turning Text Prediction into Controlled Actions

A minimal Agent is not a "chat model capable of reasoning," but a policy capable of selecting actions within a **model–Harness–environment** loop. At a minimum, tool $\mathcal{T}_j$ should be defined as

$$\mathcal{T}_j = (n_j, d_j, \Sigma_j^{\text{in}}, P_j, \Delta_j, \Sigma_j^{\text{out}}), \tag{1}$$

where n_j and d_j are the tool name and purpose description, Σ_j^{in} is the input schema, P_j is the permission/scope, Δ_j is the execution budget, such as timeout and result size, and Σ_j^{out} is the return structure. The model does not execute tools directly; instead, based on history h_t, it generates

$$a_t \in \{\text{CALL}(n_j, z_t),\ \text{FINISH}(y)\}, \qquad z_t \vDash \Sigma_j^{\text{in}}, \tag{2}$$

The Harness then performs schema validation, permission checks, timeouts/retries, actual execution, and logging, and wraps the result as observation o_{t+1} to return to the model. Toolformer investigates how models learn "when to invoke a tool, which tool to invoke, and how to use its result"[1]; ReAct interleaves reasoning, action, and environment observation into multi-step trajectories[2]. Both demonstrate that **tool use is structured action generation, whereas an Agent is a control loop that repeatedly selects actions based on feedback**.

Listing 1: A minimal search-tool contract and one invocation

```
{"tool":{"name":"search","description":"search indexed documents",
 "input_schema":{"type":"object","properties":{
   "query":{"type":"string"},"top_k":{"type":"integer","maximum":20}},
   "required":["query"],"additionalProperties":false},
 "permission":"read_only","timeout_ms":10000}}
{"action":{"name":"search","arguments":{"query":"FIM training","top_k":5}}}
{"observation":{"status":"OK","results_artifact":"sha256:9af...","truncated":false}}
```

Four types of failure must be distinguished here: invalid parameters generated by the model constitute a *policy/schema failure*; the Harness rejecting an out-of-scope action constitutes a *policy/safety failure*; a timeout in the search service constitutes an *infrastructure failure*; and a normal tool response with insufficient evidence constitutes a *task-strategy failure*. If logs uniformly record all of them as "failures," subsequent SFT, RL, and evaluation will all make incorrect attributions.

1.2 Progressive Expansion: Search Agent, Deep Research, and Code Agent

Search Agent already contains a complete Agent loop, but its environment is typically read-only, its action space is small, and state changes are limited. A usable minimal implementation is: generate a query → invoke search → select and open results → extract evidence with provenance → determine whether the information is sufficient; if insufficient, rewrite the query based on the gap, and if sufficient, generate an answer with citations. WebGPT trains models to search and navigate the web in a text-based browsing environment and requires them to collect citations supporting their answers during browsing[3]. In engineering practice, the loop should impose limits on the number of query/open operations, domains and time ranges, duplicate-URL removal, evidence freshness, and an "insufficient evidence" termination condition.

Deep research agent extends linear search into a longer research workflow: first decompose the objective into verifiable subquestions and maintain a query plan and evidence store for each branch; iteratively perform searches, reading, file parsing, or computation; then check coverage, source quality, temporal consistency, and mutually contradictory evidence; and finally synthesize a report according to claim–evidence relationships. OpenAI's public description of deep research confirms high-level capabilities such as multi-step search, interpreting webpages/images/PDFs, adjusting direction based on new information, using Python for analysis, and generating reports with citations[4]. These public materials *do not disclose the complete internal planner, concurrency scheduling, or reward implementation*; therefore, this report treats it only as a publicly observable workflow paradigm and does not infer its proprietary architecture.

The core loop is the same for all three; the difference lies in environment semantics: Search/Deep Research primarily changes internal evidence state, whereas a Code Agent modifies the external workspace, and erroneous actions can contaminate subsequent observations, break tests, or introduce security risks. A Code Agent therefore requires stronger sandboxes, versioned state, rollback, execution verifiers, and Harness auditing; this is also the starting point for the capability hierarchy

Table 1: Increasing Complexity from a Single Tool Call to a Code Agent

Form	Typical Actions	Environment and State	Termination/Verification	Primary Challenges
Single tool call	calculator, lookup, single search	Essentially no persistent state	Valid schema and a tool response	Argument generation, permissions, and exception handling
Search Agent	search, open, find, quote	Read-only webpages and short-term evidence	Question answered with traceable citations	Query rewriting, source selection, and termination decisions
Deep research	decomposition, multi-path retrieval, reading PDFs, computation, synthesis	evidence store, plan, and coverage state	Claims have evidence; conflicts/gaps are explained	Long-horizon planning, evidence governance, cost, and concurrency
Code Agent	search, view, edit, test, submit	**mutable repo, dependencies, diff, and test state**	patch passes targeted and regression verification without exceeding permissions	State recovery, long-horizon credit assignment, environment, and safety

and training loop below.

1.3 From Code Generation to a Closed Software-Engineering Loop

Traditional Code LLMs primarily learn the conditional distribution $p_\theta(y \mid x)$: generating code from a comment, function signature, or local prefix. A Code Agent instead faces a partially observable software-engineering process with continuously changing state: its input includes not only requirements, but also repository snapshots, dependency environments, historical actions, command output, and test feedback; its output is not merely code, but a sequence of actions such as searching, reading, editing, executing, rolling back, and submitting. SWE-bench instantiates the problem as "repository + Issue → a patch that passes hidden tests"; its 2,294 original tasks are drawn from real Issue/PR pairs in 12 Python repositories[5]. SWE-agent and OpenHands further show that interface design and sandboxed execution are key mediators determining whether model capabilities translate into task success[6, 7].

The existing literature has no single standard "five-layer Code Agent framework." The Agentic Issue Resolution Survey divides the task process into five stages: repo preprocessing, localization, repair, patch validation, and patch selection, while treating memory, context management, and related mechanisms as cross-cutting concerns[8]; an empirical study of Agent execution trajectories identifies localization, patch generation, and reproduction-test generation as three core links in successful workflows[9]; the Self-Evolving Coding Agents Survey distinguishes framework, memory, skills/tools, model, and workflow/topology from the perspective of system components that can evolve[10]. These taxonomies respectively answer "how the task proceeds," "which actions lead to success," and "which parts of the system evolve," but do not directly cover the full lifecycle from foundation-model training to product operations.

Therefore, the following five layers constitute **the capability stack synthesized in this report for training and engineering attribution**, rather than a standard classification copied from any one survey:

1. **Code foundation-model layer**: syntax, APIs, algorithms, explanation, completion, repair, and multilingual transfer;
2. **Repository modeling layer**: file structure, symbol/call/dependency relationships, cross-file retrieval, and long-context utilization;
3. **Interaction policy layer**: decomposing problems into multi-step actions such as localization, reproduction, modification, testing, and verification;
4. **Execution and verification layer**: reproducible environments, tool protocols, permission boundaries, testing, and verifiers;
5. **Product feedback-loop layer**: latency, cost, human takeover rate, safety, feedback recycling, and continuous evaluation.

Capability Boundaries:A long context window only expands the *visible state*; it does not imply that the model can localize dependencies, maintain a plan, or make correct modifications within it. SFT only learns the demonstration distribution; it is not equivalent to directly optimizing execution outcomes. SWE-bench resolve rate covers only a particular distribution of repository-level repair tasks; it is not equivalent to automating the full software lifecycle. RULER has shown that most long-context models degrade significantly as input length and task complexity increase[11]; Agent-

Table 2: Relationship Between This Report's Five-Layer Capability Stack and Existing Research Taxonomies

Capability Layer in This Report	Corresponding Task Process	Corresponding Agent Component/Trajectory	Additional Engineering Perspective in This Report
Code foundation-model layer	parameterized capability for repair / patch generation	model self-evolution	Attribution across pre-training, mid-training, and language/code capabilities
Repository modeling layer	preprocessing + localization	context, memory, repository exploration	repo organization, retrieval, and effective long context
Interaction policy layer	dynamic orchestration of localization–repair–validation	workflow, planning, action trace	Multi-turn policies, budgets, and credit assignment
Execution and verification layer	reproduction, validation, selection	tools, framework, verifier	harness, sandbox, environment reliability, and replayability
Product feedback-loop layer	continuous operation beyond benchmarks	memory / workflow evolution	Cost, safety, human takeover, feedback recycling, and version governance

less shows that a clear "localize–repair–validate" process can compete with complex autonomous scaffolds in some settings[12].

1.4 Unified Formalization

We formulate a repository-level software-engineering task as a decision process with environment state. At step t, the true state s_t includes the code, dependencies, workspace diff, and test state. The Agent observes only $o_t = g(s_t)$ and selects action $a_t \sim \pi_\theta(\cdot \mid h_t)$, where history $h_t = (o_1, a_1, \ldots, o_t)$. The objective is:

$$\max_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R_{\text{task}}(\tau) - \lambda_c C(\tau) - \lambda_r R_{\text{risk}}(\tau)\right], \tag{3}$$

where R_{task} comprises fail-to-pass, pass-to-pass, static checks, and a human rubric; C measures token, wall time, container, and tool-call costs; and R_{risk} represents risks from out-of-scope modifications, test gaming, supply-chain compromise, or data leakage. Equation (3) reminds us that maximizing test pass rate alone incentivizes deleting tests, hard-coding, and overfitting; regression, safety, and cost must be modeled jointly.

2 Overall Architecture: Advancing Model, Data, Environment, and Evaluation in Parallel

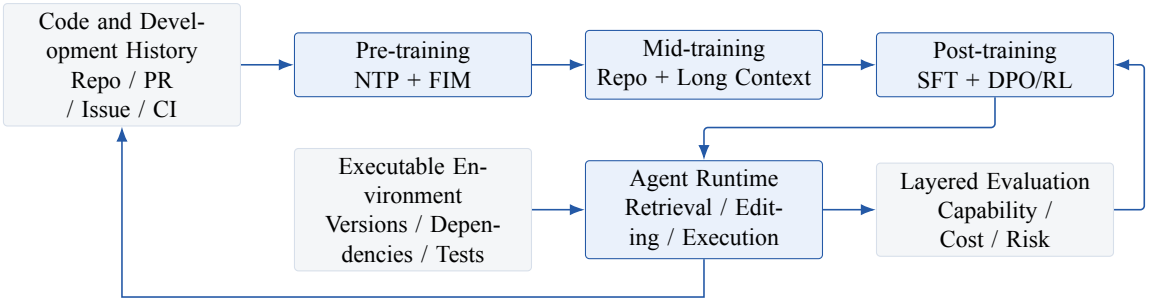

Figure 1: The four-track closed loop for a Code Agent. Training data must retain lineage through executable environments and evaluation results, and runtime trajectories must flow back into the next round of SFT, preference, or verifier data.

The key in Figure 1 is not the sequence of stages, but the synchronized advancement of four tracks: model training cannot obtain reliable rewards without execution environments; environments cannot be decontaminated without data lineage; evaluation cannot distinguish model gains without a fixed scaffold and budget; and trajectory feedback without failure types and version information will mistake accidental successes for transferable strategies.

Table 3: Independent Acceptance Surfaces Required at Each Stage

Stage	Training Objective	Primary Data	Irreplaceable Acceptance Criteria
Pre-training	Code distribution, local completion, semantic and modification priors	source code, documentation, Notebook, Issue/PR/Commit	HumanEval/MBPP, etc., FIM, per-language breakdowns, held-out loss
Mid-training	Cross-file dependencies, repository structure, long-range localization and editing	repo pack, dependency graph, PR diff, long code-related text	CrossCodeEval, RepoBench, length buckets, cross-file retrieval/completion
Post-training	Instruction following, tool protocols, decision-making, and self-repair	Multi-turn trajectories, preference pairs, executable tasks, failure feedback	tool-call validity, loop rate, patch apply, resolve rate
Runtime	Safely map policies to real actions	ACI, sandbox, index, tests, cache	latency, cost, unauthorized-action rate, human-takeover rate, auditability
Continuous evaluation	Freshness and robustness to the deployment distribution	live tasks, private regression sets, online replay	decontamination, same budget, same scaffold, temporal slices, version reproducibility

3 Pre-training: First Establish High-Quality Code and Modification Priors

3.1 Data Composition Is Not Simply "the More Pure Code, the Better"

Public technical reports provide two types of complementary evidence. OpenCoder's RefineCode contains source code and code-related Web data, and emphasizes exact + file-level fuzzy dedup, language-specific filtering, and code-related recall[13]; in the ablations reported for Qwen2.5-Coder, the combination of 70% Code, 20% Text, and 10% Math outperforms a pure-code setting, but this is a reported result under a specific model and data scale and cannot be directly extrapolated as a universally optimal mixture[14]. StarCoder2/The Stack v2, meanwhile, provides more transparent license detection, Software Heritage lineage, and a multi-source composition including Issue, PR, Notebook, and documentation[15].

The pre-training pool should be partitioned into the following mutually exclusive or traceable logical domains:

- **Source Code**: mainline snapshots, historical versions, tests, configuration, build scripts, IDL files, and database schema files;
- **Code-related Text**: official documentation, tutorials, README files, design documents, API reference materials, and technical Q&A;
- **Development History**: Commit message/diff records, PR discussions, Issue records, review comment records, and CI logs;
- **Executable/Synthetic**: compilable samples, synthetic problems with unit tests, and execution-filtered text-to-code data;
- **General/Math**: controlled general-purpose data for maintaining instruction comprehension, reasoning, and non-code knowledge.

Do not allow multiple domains to count the same token occurrences at the physical layer. For example, if the before/after files in a PR diff also enter source code as snapshots, content hash values and commit lineage must be established, with the ability to report at least counts for unique token, repeated token, and actual trained token.

3.2 Cleaning, Filtering, and Deduplication

Code data quality requires simultaneously addressing four questions: "Does the file resemble code?", "Does the code have learning value?", "Is the data duplicated?", and "Do the license/privacy constraints permit its use?" The recommended pipeline is: format identification → exact dedup → basic cleaning → language-specific parsing → quality/value filtering → fuzzy dedup → contamination removal → mixture sampling. Performing exact dedup early can substantially reduce downstream computation; OpenCoder reports a large number of exact duplicate files in its raw pool and finds in its own ablation that file-level fuzzy dedup outperforms repo-level dedup[13]. This result shows that "repo as an organizational unit" and "repo as a deduplication unit" should not be conflated: preserving repository structure does not require using entire repositories for similarity decisions.

For each language family, separately track: parse success rate, generated/vendor/test/config identification rate, rule trigger rate, human-estimated false-positive removal rate, retained token count, dedup ratio, and benchmark contamination

hit count. For code-specific processing, particular care should be taken not to erroneously remove the following: symbol-dense C/C++ macros, short but critical interface/type declarations, template-generated code, test fixture data, SQL/configuration, and cross-language binding code.

Let the sampling weight of the d-th data domain be w_d, sample quality be q_i, and temperature be T; constrained weighted sampling can be used:

$$p(i) = \frac{w_{d(i)} \exp(q_i/T)}{\sum_j w_{d(j)} \exp(q_j/T)}, \qquad \mathcal{L}_{\text{pre}} = -\sum_i p(i) \sum_t m_{i,t} \log p_\theta(x_{i,t} \mid x_{i,<t}), \tag{4}$$

Here, $m_{i,t}$ allows license declarations and template boundaries to be masked, or loss to be computed only on the FIM middle. An overly low T excessively concentrates high-scoring templates and reduces diversity; w_d should be determined through equal-token small-model ablations rather than by raw data volume.

3.3 Training Objectives: NTP, FIM, and Modification Modeling

Next-token prediction (NTP) remains foundational, but a Code Agent additionally needs to learn to "insert and modify within existing context." FIM splits the original sequence into prefix, middle, and suffix, and performs autoregressive training with

$$\tilde{x} = [\text{PRE}; x_{<l}; \text{SUF}; x_{>r}; \text{MID}; x_{l:r}] \tag{5}$$

DeepSeek-Coder uses FIM on project-level code and extends the training window to 16K[16]; Qwen2.5-Coder first performs 8K file-level NTP/FIM, then 32K repo-level FIM, and supports a longer inference window through YaRN[14]. In engineering practice, FIM rate, PSM/SPM format, middle length, cross-file target position, and loss mask should be jointly ablated, avoiding tuning only on HumanEval-FIM.

Development history provides a third type of supervision: intent + before → after/diff. OctoPack organizes Git commit records into natural-language instructions and modification pairs, demonstrating that commit data can serve as cross-language repair/explanation signals[17]; Clean-PR further uses rigorously filtered real PR records for repo-level mid-training and emphasizes patch applicability and decontamination[18]. Pre-training should include only high-confidence local modification sequences, leaving complex PRs to mid-training to reduce input noise and target ambiguity.

4 Data Collection: Code, PR/Issue, and Trajectories Must Share Lineage

4.1 Unified Data Object

A sustainable data platform should not maintain disconnected "pre-training JSONL," "SFT conversations," and "evaluation instances." It is recommended to use repository snapshot as the root object, linking downward to files, symbols, Issue records, PR records, Commit records, environments, tasks, trajectories, and evaluation results. Minimum lineage should include at least: repo URL, license, base commit, target commit, file hashes, collection time, data split, build image digest, test commands, model/scaffold version, and contamination labels.

Listing 2: Recommended Unified Task and Trajectory Record (Illustrative)

```
{
  "task_id": "repo@base_commit#issue_or_synthetic_id",
  "repo": {"url": "...", "base_commit": "...", "license": "..."},
  "problem": {"text": "...", "source": "issue|pr|synthetic"},
  "environment": {"image_digest": "sha256:...", "test_cmd": "..."},
  "oracle": {"patch_hash": "...", "fail_to_pass": [], "pass_to_pass": []},
  "trajectory": [{"observation": "...", "action": "...", "result": "..."}],
  "outcome": {"resolved": false, "failure_type": "localization"},
  "provenance": {"collected_at": "...", "split": "train", "contamination": []}
}
```

4.2 Real Tasks, Synthetic Tasks, and Trajectory Data

Real Issue/PR data have the advantage that intent, discussion, and modifications all originate from the development process, but they are subject to association errors, environment decay, hidden dependencies, merge noise, and insufficient tests. Synthetic tasks are easier to scale and control for difficulty, but may learn generator-specific artifacts. SWE-Gym provides 2,438 real Python tasks with executable environments and releases successful trajectories; its rejection-sampling SFT shows that a small number of high-quality multi-turn trajectories can substantially reduce loops and empty patches[19]. SWE-smith first creates a shared environment for each repository, then synthesizes tasks using function rewriting, programmatic mutation, composite bugs, and PR mirror, reporting approximately 50K instances across 128 repositories and training a model with 5,016 expert trajectories[20]. R2E-Gym uses test generation and commit back-translation to construct 8.7K-scale environments and studies the complementarity of execution-based and non-execution-based verifier methods[21]. SWE-rebench, meanwhile, targets continuous collection and fresh evaluation, releasing more than 21K interactive Python tasks and explicitly discussing contamination[22].

Four engineering rules can be derived from these works:

1. **Environment first**: prove that the base commit can be installed and that baseline tests are stable before generating or incorporating tasks;
2. **Dual validation**: the gold patch should achieve F2P while preserving P2P; rerun tests to eliminate flaky test cases;
3. **Collect successes and failures**: successful trajectories are used for behavior cloning, while failed trajectories are used for preference, verifier, failure classifier, and curriculum;
4. **Per-instance quotas**: excessive successful sampling from the same easy task creates easy-task bias; apply a task/repo cap and report the number of trajectories after deduplication.

4.3 Trajectory Quality and Failure Labels

Trajectories should be decomposed into diagnosable stages rather than having only a final reward: problem understanding, file localization, symbol localization, error reproduction, solution generation, patch apply, local testing, regression testing, and termination determination. The following failure labels should be retained: env_setup, localization, reasoning, edit_syntax, test_generation, regression, loop, budget, unsafe_action, and invalid_task. Labels may be jointly produced from rules, execution signals, and an LLM judge, but the judge may only provide soft labels and cannot replace test facts.

Trajectory value should not be filtered solely by success. It can be defined as:

$$v(\tau) = \alpha r_{\text{exec}} + \beta r_{\text{progress}} + \gamma d_{\text{novelty}} - \delta c_{\text{tokens}} - \eta r_{\text{risk}}, \tag{6}$$

where r_{progress} can consist of dense signals such as correct localization, successful reproduction, and a reduction in the number of failing tests; d_{novelty} approximates task/repo/action n-gram or embedding diversity. Equation(6) is an engineering recommendation, not a unified definition from the published literature.

5 Mid-training: Transitioning from Single-File Models to Repository-Level Editing

5.1 Three Levels of Repo-level Organization

Repository data can be organized at no fewer than three granularities:

1. **Random concatenation within a repository**: preserves repo boundaries and each file path but does not explicitly encode dependencies; it has the lowest cost and is a necessary baseline;
2. **Structure-aware concatenation**: orders or samples by import, call, inherit, test-to-source, and build dependency relations;
3. **Task-conditioned organization**: constructs a local subgraph around the target symbol/issue and adds hard negative examples and oracle context.

DeepSeek-Coder and StarCoder2 demonstrated that project/repository-level sequences can be used to train open code models[16, 15]; however, "placing files from the same repo into the same window" does not automatically prove that the model has learned dependency relations. CrossCodeEval explicitly requires cross-file context and covers Python, Java, TypeScript, and C#[23]; RepoBench separately evaluates retrieval, completion, and pipeline[24]. Therefore, multi-level

references (DFS/BFS), file-level, same-repo shuffled, and dependency-aware variants must be compared with equal token counts, language mixtures, and context lengths.

The recommended 8K/32K pack consists of the following units: local context from the target file + direct dependencies + tests/callers + documentation/Issue + hard negatives. If the window is insufficient, declarations, signatures, types, call sites, and tests should be retained in preference to complete implementations. The retrieval strategy used by the online runtime should match the context distribution in mid-training; otherwise, relying on oracle context during training but retrieval with low recall during inference will create a pronounced distribution shift.

5.2 PR/Issue as mid-training Signals

PR/Issue data connects "user intent–repository state–modification–discussion–testing" and is a natural bridge for transitioning from code models to editing models. The following multitask formulation is recommended:

- Issue → changed files / symbols (localization);
- Issue + local tree → inspection actions (retrieval strategy);
- before + intent → diff / str_replace (modification);
- diff + source → tests (unit test generation);
- candidate patch + logs → critique / repair (self-repair);
- PR discussion → review decision / revised patch (code review).

The latest published Clean-PR results support using rigorously filtered and decontaminated PR data whose patch is applicable as mid-training signals, but the specific gains depend on the model, filters, and evaluation scaffold used[18]. In engineering practice, file recall@k, symbol recall@k, patch apply rate, edit exactness, and held-out trajectory loss should be measured before end-to-end resolve; this distinguishes "learning to localize," "learning to edit," and "scaffold remediation."

6 Long-Context Extension: The Goal Is Effective Utilization, Not a Larger Configuration Value

6.1 Position Extension, Data Adaptation, and System Cost

RoPE maps the m-th position to a dimension-wise rotated representation. For the i-th two-dimensional channel, its phase can be written as

$$\phi_i(m) = m\theta_i, \qquad \theta_i = b^{-2i/d}. \tag{7}$$

Beyond the training length, $\phi_i(m)$ enters an OOD region. YaRN reduces the cost of context-extension training through frequency partitioning, interpolation, and attention scaling[25]; LongRoPE further searches for nonuniform interpolation and adopts progressive extension[26]. These methods address position representations and training stability; they do not guarantee that the model possesses repository-level multi-hop reasoning capabilities. LongAlign's findings also indicate that long-context models require instruction data of comparable length and must account for packing and loss weighting[27].

The compute and activation costs of standard full attention scale approximately with length as

$$C_{\text{attn}} = \mathcal{O}(L^2 d), \qquad M_{\text{KV}} = \mathcal{O}(L\, n_{\text{layer}}\, d_{\text{kv}}), \tag{8}$$

Therefore, extending from 32K to 128K is not simply a matter of changing rope_theta: sequence parallel/context parallel, FlashAttention, length bucketing, packing, checkpointing, KV cache strategy, and inference budget must be jointly designed. A Code Agent should also ask: is it truly necessary to put the entire repo into the window, or is 32K–64K retrieval with high recall + iterative inspection more cost-effective?

6.2 Recommended Long-Context Curriculum

The training mixture cannot use only the maximum length. The token proportion of each batch bucket should be specified explicitly; retaining 30%–60% short-sequence replay is a recommended initial exploration range, to be adjusted experimentally. RULER shows that vanilla needle accuracy may conceal degradation in multi-hop and aggregation capabilities[11], so long-code evaluation must also include cross-file reference-chain depth, number of distractor files, target position, and repo size buckets.

Table 4: Long-context training curriculum and gates

Stage	Length distribution	Data organization	Gate
L0 Preserve capabilities	Primarily 4K–8K	file-level NTP/FIM + general/math replay	No significant regression on the short-context benchmark
L1 Position adaptation	Mixture of 8K/16K/32K	same-repo pack + long documents	Length-bucketed PPL, passkey, multiple needle tests
L2 Structural learning	16K–64K	dependency-aware, multi-level references, target-aware	CrossCodeEval/RepoBench improve with effective context
L3 Instruction alignment	A small amount of 32K–128K	Long Issue examples, directories, trajectories, multi-file diff	No degradation in long-range localization, plan retention, or tool protocols
L4 Inference optimization	Real-world distribution	Retrieval compression, KV/reuse, iterative browsing	Lower latency/token cost at the same success rate

7 Post-training: From “Answering” to “Acting and Verifying”

7.1 SFT: First Learn Stable Protocols and Basic Strategies

SFT data should cover at least three categories: atomic tool skills, structured workflows, and unconstrained multi-turn trajectories. Atomic skills include search/view/edit/test/git diff; structured workflows include localization–modification–verification; unconstrained trajectories cover dynamic revision based on execution feedback. CodeAct unifies the action space with executable Python and releases 7K multi-turn interaction examples, demonstrating that the “action representation” itself affects Agent learning[28]. SWE-agent emphasizes the impact of ACI on navigation, editing, and testing behavior[6].

The training mask should distinguish system/tool schema, observation, reasoning, and action. If the product does not output explicit chains of thought, auditable short plans, state summaries, and action rationales can serve as training targets without relying on lengthy unconstrained reasoning. Key SFT metrics include tool-call parse rate, action validity, empty patch, loop rate, average turns, invalid-read ratio, and test-before-finish ratio.

7.2 Preference Optimization and verifier

Preference pairs can be constructed from multiple trajectories for the same problem: success is preferred over failure; no regression is preferred over merely passing new tests; a smaller, well-explained patch is preferred over a broad rewrite; and, for equal success, lower cost is preferred over higher cost. DPO can learn relative preferences without online sampling, but if preference labels are produced primarily by a judge, the model will learn the judge’s stylistic and length biases. Execution facts, a human rubric, and model scores should be combined, and reward hacking should be evaluated separately.

An Outcome verifier estimates the following for trajectory τ and patch P

$$s_\phi(\tau, P) = P_\phi(\text{YES} \mid \text{task}, \tau, P), \tag{9}$$

for Best@k selection or process pruning. SWE-Gym trains an outcome-supervised verifier using successful/failed trajectories[19]; R2E-Gym finds that execution-based and execution-free verifier methods each have blind spots and are more effective when combined[21]. Therefore, a production verifier should jointly incorporate regression tests, targeted test generation, static analysis, diff risk, and learned score rather than a single reward model.

7.3 Why Code Agent RL Is Not Ordinary Single-Turn RLVR

RL is the key module that moves a Code Agent from “imitating successful trajectories” to “actively exploring based on environment feedback,” but software engineering tasks simultaneously have four characteristics: **mutable state, extremely long trajectories, sparse rewards, and expensive, failure-prone environments**. RLVR for mathematics/competitive programming typically needs only to score one complete output; a Code Agent receives a terminal reward from hidden tests only after dozens of rounds of tool interaction spanning tens of thousands to over one hundred thousand token. Published experiments on Long-Context Multi-Turn SWE RL formalize this difference as a stateful MDP and point out that broadcasting one terminal advantage across the entire trajectory introduces severe credit-assignment noise[29].

Therefore, three prerequisites should be satisfied before training: first, the initial SFT/RFT policy must already have nonzero pass@k on training problems; otherwise, every rollout in the group is zero and group-relative advantage is unusable;

second, the gold patch, empty patch, and repeated runs must verify evaluator determinism; third, infrastructure failures such as container startup, dependency installation, and flaky tests must be separable from policy failure. The direct objects of RL improvement are not the “total amount of code knowledge,” but the closed-loop strategies of *search order, experiment selection, failure recovery, termination, and submission*.

7.4 Four Technical Routes Validated in Practice

7.4.1 Route A: proxy-reward RL Based on Software Evolution Data

SWE-RL constructs issue, pre-change context, and oracle patch examples from GitHub Issue/PR, uses sequence similarity between the predicted patch and oracle patch as a continuous reward, and optimizes with GRPO[30]. This method does not require launching a complete repository environment, offers high throughput, and scales easily; continuous similarity is also denser than a discrete reward based on “whether there is an exact match.” Its limitations are equally clear: patch text similarity is not equivalent to semantic correctness and penalizes a valid patch that differs from the developer’s implementation; moreover, its primary training task provides the relevant code context, reducing the difficulty of localization and multi-turn execution faced by a real Agent. It is better suited to large-scale reasoning/editing warm-up and should not serve as the final agentic reward.

7.4.2 Route B: Long-Trajectory RL in Real Execution Environments

Execution-based RL directly judges a patch with F2P/P2P tests. Golubev et al. use RFT cold-start and a modified DAPO in a multi-turn environment with 131K context: they sample 10 complete trajectories per problem, normalize the terminal reward within each group, discard groups with zero advantage, and add a trajectory-length penalty; under the same scaffold, this improves from an RFT baseline of 20% to 39% on SWE-bench Verified[29]. The paper also warns that post hoc truncation of overlength trajectories creates an inconsistency between the rollout policy and training log-prob, violating the importance sampling assumption.

SWE-Master provides more engineering-oriented stabilization strategies[31]: first perform SFT on approximately 60K long trajectories, then perform GRPO in Docker environments; maintain exploration with leave-one-out advantage, fixed-length normalization, clip-higher, and removal of KL; force budget-exhausted trajectories to submit the current patch and discount reward according to stop reason; directly mask container/service failures so they do not enter the policy update; and explicitly return the remaining budget in every observation. Its key lesson is not any single hyperparameter, but that **termination semantics and infrastructure semantics must be incorporated into the RL data model**.

7.4.3 Route C: Reducing Sparsity with guidance or Sub-capability Decomposition

Agent-RLVR lets the Agent first attempt tasks independently, then provides a plan, error corrections, or environment-interaction hints for failed trajectories, and updates with verifiable rewards after rerollout; guidance appears only during training sampling and is removed at inference time[32]. Its Qwen2.5-72B-Instruct improves from 9.4% to 22.4% on SWE-bench Verified, and reaches 27.8% by further training a reward model on the same batch of trajectories for candidate ranking. This shows that the teacher’s value can lie in *causing successful samples to appear in sparse-reward regions*, rather than necessarily distilling complete answers into the student; however, one must check whether the model depends on the guidance style and whether the teacher leaks gold information.

Another approach is to first train atomic policies that permit dense scoring. CodeScout defines code localization as set prediction at three granularities—file/module/function—uses the sum of their three F1 scores as the reward, and has the Agent search using only a Unix terminal and submit through a structured finish tool[33]. This design changes the reward from “whether the final patch is completely correct” to “how accurately relevant locations are found,” while avoiding brittle string parsing. Subskill RL should be validated in tandem with end-to-end RL: an increase in localization F1 constitutes effective transfer only if it improves resolve rate under a fixed repair model and fixed budget.

7.4.4 Route D: Self-Play Generates an Executable Curriculum of Increasing Difficulty

Self-play SWE-RL has the same policy alternate between the roles of bug injector and solver: the injection side generates bug patches and test-weakening patches from real repositories and filters non-executable samples through consistency checks; the solver repairs them under hidden-test constraints; the injection reward encourages bugs that are “valid, non-trivial, yet still solvable,” rather than simply as difficult as possible[34]. Without relying on human Issue/PR annotations, the method reports gains of 10.4 and 7.8 points on SWE-bench Verified and SWE-Bench Pro, respectively. Its key risk is degeneration of the self-play distribution: the injector may learn artificial patterns that are easy to score but unlike real defects, so bug type, modification scope, language/repo distribution, and transfer to real Issues must be monitored.

Table 5: Signals, Advantages, and Major Biases of Code Agent RL Routes

Route	Primary reward	Problems suited for	Major biases/gates
Proxy patch RL	oracle patch similarity, format	Low-cost reasoning/editing warm-up	Penalizes equivalent patches; must be rechecked with execution evaluation
Execution RL	F2P/P2P, regression and termination discount	End-to-end search–edit–verify policy	Environment throughput, flaky tests, sparse rewards, and long-horizon credit
Guided / skill RL	guided success, localization F1, process milestones	Cold start and bottleneck capabilities such as localization/testing	guidance leakage; decoupling of proxy and resolve rate
Self-play RL	bug validity, target difficulty, repair success	Expanding task distribution and automated curriculum	Collapse of synthetic-defect patterns; requires a real-task transfer gate

7.5 Actionable Recipes for Rewards, Optimization, and Credit Assignment

An execution reward should not retain only a success bit. It is advisable to retain an auditable vector and combine it on the training side:

$$\begin{aligned} r(\tau) = \alpha r_{\text{F2P}} + \beta r_{\text{P2P}} + \gamma r_{\text{progress}} + \eta r_{\text{submit}} \\ - \lambda_1 c_{\text{turn}} - \lambda_2 r_{\text{unsafe}} - \lambda_3 r_{\text{test tamper}} - \lambda_4 r_{\text{scope}}. \end{aligned} \tag{10}$$

r_{progress} may come only from evidence that is difficult to game, such as fewer compilation errors, a larger subset of target tests passing, a reproduction test changing from pass to fail, or convergence of static type errors; it must not directly reward superficial actions such as "invoked tests" or "read files." r_{submit} distinguishes active submission from forced submission after budget exhaustion; r_{scope} constrains unrelated files and diff expansion. Deleting/relaxing tests, hard-coding examples, modifying the grader, bypassing dependencies, or writing state outside the environment should all trigger a hard zero or sample removal via an independent rule-based detector.

For optimization, critic-free methods such as GRPO/GSPO/DAPO can serve as a starting point, but group success diversity must be recorded: if every sample in a group succeeds or every sample fails, the update signal is zero; if difficult problems remain all-zero for a long time, one should increase the number of rollouts, add guidance/a curriculum, or fall back to rejection fine-tuning, rather than blindly extending training. For ultra-long trajectories, broadcasting a single trajectory advantage to all tokens is only a baseline. Further options include: (1) branching rollouts from a shared prefix to compare the counterfactual value of subsequent actions; (2) using verifiable milestones to generate step/segment rewards; (3) training a critic/value head for local advantage; and (4) computing loss only on model action tokens, while masking tool observations and infrastructure anomalies. The denser the process reward, the more carefully one must check whether it induces "progress farming" rather than task completion.

7.6 RL Systems, Data Lineage, and Experimental Attribution

The training system must include at least an environment pool, rollout workers, a trajectory/event store, a reward/evaluator, a trainer, and a checkpoint broadcaster. Each sample must be bound to a repo commit, container digest, test patch, harness/scaffold commit, model revision, sampling seed, stop reason, reward components, and artifact URI. Asynchronous rollout can improve GPU/CPU utilization, but trajectories generated by old checkpoints increase off-policy staleness; the policy version should be recorded and a maximum lag set, rather than mixing all trajectories as if they came from the same distribution.

The minimal causal experiment matrix should fix the base/SFT checkpoint, task split, harness, context, turn/token budget, and total number of rollouts, and compare: SFT/RFT, offline preference, proxy-reward RL, execution RL, execution RL + guidance, execution RL + process signal, and best-of-k + verifier without parameter updates. Beyond resolve rate, primary metrics should also report pass@k, group nonzero rate, reward/entropy, active-submit rate, forced-submit success, environment mask rate, mean turns/tokens, diff size, regression/privilege-violation rate, and unseen-repo transfer. An improvement can be attributed to policy learning only if it outperforms SFT and test-time scaling under the **same sampling budget** and retains gains on held-out repos.

8 Agent Runtime: Enabling Models to Work Through Controlled Interfaces

8.1 A Minimal Yet Sufficient Tool Surface

The basic toolset should be limited to: directory/symbol search, segmented file reading, structured editing, terminal execution, testing, diff/status, and final submission. Tools should return stable, parseable, and length-bounded observations; editing tools must verify that old text matches uniquely or operate through patch apply; the terminal must restrict network, filesystem, and process permissions. OpenHands places the terminal, editor, browser, and other tools in a sandbox platform[7], but enterprise code settings additionally require secret redaction, dependency-source allowlists, audit logs, and human approval gates.

The runtime should not inject the "entire repo" by default. More robust context management includes a repo map, symbol graph, hybrid BM25/embedding retrieval, compression of recent observations, a persistent scratchpad, and a diff-aware cache. On each turn, the Agent should receive the current objective, known facts, workspace state, and remaining budget, rather than an ever-growing concatenation of raw history.

8.2 Harness: The Control Plane Connecting Model Policies to the Executable World

A harness is not a scaffold, nor is it an evaluation harness. The scaffold defines the prompt, tools, and action style visible to the model; the agent harness is responsible for reliably running a task as a complete episode; the environment provides repository and execution isolation; and the evaluation harness applies the patch, runs standard tests, and scores the result after the episode ends. A result should be explicitly written as

$$Y = f(M, S, H, E, V, B, \xi), \tag{11}$$

where M is the model checkpoint, S is the scaffold/prompt, H is the agent harness, E is the environment image, V is the evaluator/verifier, B is the budget, and ξ is sampling randomness. Reporting only "model name + SWE-bench score" cannot reproduce the remaining variables in Equation (11).

The Agent harness should implement an explicit episode state machine: INIT → OBSERVE → MODEL_STEP → PARSE/POLICY → EXECUTE → UPDATE, entering a terminal state on success, failure, timeout, budget exhaustion, privilege violation, or human interruption. Every transition must be idempotent or make it possible to determine whether it has already executed, preventing model/API retries from causing the same write operation to occur repeatedly.

Table 6: Core Modules and Acceptance Criteria of a Code Agent Harness

Module	Responsibility	Failures and metrics that must be tested
Episode Orchestrator	State machine, termination conditions, retries, checkpoint/resume	Duplicate execution, infinite loops, recovery consistency, terminal reason distribution
Model Adapter	Unified chat/response/tool-call, streaming output, and token accounting	schema differences, truncation, rate limit, actual input/output tokens
Context Manager	history, repo map, retrieval, compression, cache, and context budget	Information loss, context overflow, stale cache, effective context hit rate
Action Parser/Policy	Action parsing, parameter validation, allow/deny/approval, deduplication	parse rate, invalid action, privilege-violation blocking, number of approvals
Environment Adapter	local/container/remote sandbox, workspace and process lifecycle	base commit drift, zombie processes, network/filesystem boundary violations, resource leaks
Tool Runtime	Timeouts, output truncation, and error mapping for search/view/edit/shell/test/git	command timeout, partial output, patch conflicts, tool error rate
Budget Controller	Limits on turns, tokens, wall time, cost, CPU/GPU/IO	Over-budget actions, budget estimation error, success-cost distribution
Event Store	append-only events, artifacts, diffs, logs, traces, and replay	Missing events, sensitive information, trace replayability rate, version-field completeness
Evaluator Bridge	Generate standard predictions, submit to the evaluation harness, collect fine-grained results	patch serialization, F2P/P2P, gold sanity, evaluation timeout

Harness design complexity should match the research question. SWE-agent is suitable for studying toolsets, history processors, and ACI; mini-SWE-agent instead uses a linear message history, independent shell actions, and minimal control flow, returning the research focus to the model itself[35]. Neither is absolutely superior: a complex harness facilitates

guardrails, retrieval, and recovery, whereas a minimal harness is better suited as a model-capability baseline and reduces scaffold overfitting.

The evaluation side must be decoupled from the agent harness. The official SWE-bench evaluation harness uses layered Docker images, executes setup, patch application, testing, grading, and reporting, and provides cache, timeout, and log controls[36]. Internal evaluation should likewise first perform a harness sanity check with the gold patch, then test empty-patch/incorrect-patch negative controls; otherwise, environment or grader failures will be misrecorded as model failures. Each run must freeze at least the following manifest:

Listing 3: Minimal Reproducibility Information in a Harness Run Manifest

```
run_id, task_set_version, model_id, model_api_revision,
scaffold_commit, harness_commit, prompt_hash, tool_schema_hash,
environment_image_digest, evaluator_commit, retriever_index_digest,
context_limit, max_turns, max_tokens, timeout_s, sampling_seed,
network_policy, permission_policy, cache_policy
```

For training-data production, the harness must also support deterministic replay and counterfactual replay: the former replays the model/actions with frozen observations to check parser and state-machine consistency; the latter substitutes the model or one action and compares subsequent branches. If observations contain nondeterministic state such as time, network, or concurrency, the original events and external artifacts should be saved rather than assuming that commands can reproduce the same output.

8.2.1 Claude Code: Reconstructing a Harness Architecture Profile from Public Interfaces

Claude Code's complete internal implementation has not been disclosed, so product behavior cannot be reverse-engineered into a definitive private module diagram. What can be confirmed from official documentation and engineering articles is a set of **public control planes**: the model selects tools in a loop driven by environment feedback; the Claude Agent SDK exposes the same tools, context management, and permission primitives as Claude Code; and project instructions, memory, compaction, subagents, hooks, MCP, permissions, and sandbox each assume distinct control responsibilities[37, 38, 39]. From this, Table 7 can be derived; it is an engineering mapping of public capabilities, not a reverse-engineered conclusion about the closed-source implementation.

Table 7: Claude Code's Public Harness Control Planes and Transferable Design Principles

Control plane	Public Claude Code mechanism	Harness responsibility	Implications for a Custom Code Agent
Context/memory	Layered instructions in CLAUDE.md, auto memory, compaction	Inject project conventions, compress history, retain reusable facts across sessions	Store instructions, working memory, and long-term memory separately and record provenance[40]
Tools/extensions	Read/Edit/Bash, MCP, Skills	Unified tool schema, observation truncation, and external connections	Separate tool capability from permissions; treat external content as untrusted input
Deterministic control	lifecycle hooks, settings, finish/stop conditions	Formatting, auditing, blocking modifications to protected files, reinjection after compaction	Put hard constraints in hook/policy rather than relying on prompt compliance[41]
Task isolation	Independent context and tool/-model/permission configuration for subagents	Isolate noisy retrieval/testing and return structured summaries	delegation must record the input summary, agent configuration, and handoff artifact[42]
Security boundary	permissions evaluate tools first; Bash is then subject to OS sandbox filesystem/network constraints	allow/ask/deny, least privilege, egress and workspace isolation	Layer policy guards with OS enforcement; neither can replace the other[43]
Recoverable execution	session resume, checkpoint, structured handoff	Recover state and unfinished tasks across context/session boundaries	Use files/events as the source of truth and verify the environment after recovery rather than trusting the summary

The public evolution of long-task Harnesses is also highly instructive. Anthropic's early approach used an initializer agent to establish the environment, task list, and progress file, then had a coding agent perform incremental work in each

session and leave a handoff artifact, addressing memory loss after context reset[44]; a later application-building approach expanded this into planner–generator–evaluator, with the generator and evaluator negotiating a testable contract before each sprint and the evaluator performing acceptance checks using browser/execution feedback[45]. These are *example Harnesses built on the Claude Agent SDK* and should not be described as Claude Code's default internal topology. Together, they show that critical state for long-horizon autonomy should reside in the repo, task table, test contract, and event log, rather than only in the chat context.

Security likewise embodies two layers of control. The public Claude Code sandbox applies OS-level filesystem constraints to Bash subprocesses using macOS Seatbelt or Linux bubblewrap, and enforces domain egress control through a proxy outside the sandbox; permissions decide allow/ask/deny before a tool runs, and the two have different scopes of coverage[43]. The public architecture of Auto mode further adds an input-side prompt-injection probe and an output-side tool-call classifier, with subagents recursively passing through the same pipeline[46]. For enterprise Code Agents, an equivalent implementation should at minimum distinguish: **model propensity constraints, Harness policy, strong OS/-container isolation, and external connector permissions**, and use attack replays to separately test whether each layer is genuinely effective.

Harness Design Boundaries:A Harness can significantly raise the task success rate of a particular model, but it encodes assumptions about "what the model currently cannot do." After a model upgrade, planner, enforced stages, additional verifier, and context reset should each be removed in turn for ablation; if the score does not decrease, that complexity should be deleted. Conversely, any experiment claiming improved model capability should first be retested under both minimal harness and production harness profiles, to avoid misattributing scaffold/harness overfitting as checkpoint gains.

8.3 Degree of Autonomy and Workflow Selection

Different applications should choose different autonomy levels:

- **L0 Suggestions**: completion, explanation, and review comments, with no direct repository writes;
- **L1 Controlled Editing**: generate a patch and execute it after user confirmation;
- **L2 Sandbox Tasks**: autonomously search, edit, and test, but without access to production resources;
- **L3 Workflow Agent**: may create branches, submit PRs, and respond to CI, with approval for critical actions;
- **L4 Long-Horizon Autonomy**: cross-task planning and continuous operation, suitable only for highly verifiable, strongly isolated scenarios.

Agentless results show that for tasks whose localization, repair, and verification can be clearly decomposed, a fixed pipeline may be cheaper and more interpretable[12]. Autonomy is therefore not the default objective; it should be opened up level by level only when dynamic environments and branching decisions demonstrably provide benefits.

9 Evaluation System: Models, Retrieval, Scaffolds, and Budgets Must Be Decoupled

9.1 Four-Level Evaluation Matrix

Table 8: Layered Evaluation Matrix for Code Agents

Level	Representative Tasks/Benchmarks	Core Metrics	Interpretive Boundaries
Function-level	HumanEval, MBPP, LiveCodeBench, BigCodeBench, DS-1000	pass@1/pass@k, execution correctness, library-call coverage	Measures syntax, algorithms, instruction following, and API use; does not represent cross-file localization[47, 48, 49, 50]
Cross-file	CrossCodeEval, RepoBench, FIM/repair suites	EM, Edit Similarity, CodeBLEU, retrieval recall@k	Results with/without oracle context and per-language breakdowns should be reported; does not directly represent multi-turn Agents[23, 24]

Table 8 (continued)

Level	Representative Tasks/Benchmarks	Core Metrics	Interpretive Boundaries
Repository tasks	SWE-bench Verified/Multilingual, SWE-bench-Live, SWE-rebench	resolve rate, patch apply, F2P/P2P, cost	Highly dependent on environment, scaffold, budget, and contamination; versions must be fixed[5, 22, 51]
Product closed loop	Private Issues, code review, migration, test generation, CI repair	acceptance rate, time-to-merge, revert, human takeover, risk, cost	Closest to business value, but requires normalization by task difficulty, user, and repository distribution

The without-replacement estimator for pass@k is[47]

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \tag{12}$$

where n is the number of samples and c is the number of correct samples. For Agents, pass@1, pass@k, and verifier-selected Best@k should be reported simultaneously; they correspond respectively to single-run policy performance, the upper bound from diversity, and selector capability, and must not be conflated as the same score.

9.2 Fair Comparison and Contamination Control

End-to-end comparisons should at minimum fix: base model checkpoint, context limit, prompt/scaffold, tool set, retriever, maximum turns, token/dollar budget, temperature, container image, concurrency, and retries. Training experiments must also fix total training tokens, language mixture, number of sampling passes, sequence-length distribution, and optimizer steps. If a repo-level scheme is trained for more epochs or more actual tokens, improvements in loss/benchmarks cannot be attributed to the organization strategy.

A four-layer parallel approach to contamination control is recommended: repo exclusion; file/patch exact hash; code 10–15 gram or MinHash similarity; Issue/description embedding/Jaccard. Temporal splits should be doubly tagged by Issue/PR creation time and model data cutoff. LiveCodeBench mitigates contamination in static code benchmarks by continuously collecting new competition problems[48]; SWE-rebench and SWE-bench-Live extend "continuous collection of fresh executable tasks" to software-engineering Agents[22, 51]. Fresh evaluation is not automatically contamination-free; repository history, derived patches, and base-model exposure must still be checked.

9.3 Recommended Dashboard

At minimum, display the following simultaneously:

- **Capability**: resolve, F2P, P2P, file/symbol recall@k, tool-call validity;
- **Efficiency**: tokens/task, turns/task, wall time, GPU/CPU/container hours, cost per successful task;
- **Behavior**: loop, empty patch, test execution, rollback, invalid actions, and context utilization;
- **Quality**: diff size, regressions, static-analysis warnings, review acceptance rate, revert rate;
- **Data**: unique/actual token, dedup ratio, repo/language/task diversity, environment build rate;
- **Risk**: secret exposure, unauthorized access, dependency supply chain, test tampering, and license violation.

10 Application Scenarios: Choose Deployment Targets by Verifiability and Risk

Initial deployments should target scenarios with clear rewards, isolated environments, recoverable failures, and manageable human review costs. Issue-to-PR is well suited to research, but the first production deployments are often localization, test generation, or controlled patches, because these make it easier to decouple model gains from system risk. For security repairs, dependency upgrades, and CI, network access and release permissions should be placed in independent executors so that the language model does not directly hold long-lived credentials.

Table 9: Application Scenarios, Training Signals, and Deployment Thresholds

Scenario	Key Capabilities/Data	Primary Evaluation	Recommended Autonomy Level
IDE completion and editing	FIM, local context, user acceptance/undo	acceptance rate, edit survival, latency	L0–L1
Repo Q&A and localization	repo map, symbol graph, Issue-to-files	recall@k, answer citability	L0–L1
Bug repair/Issue-to-PR	PR/Issue, execution trajectories, regression tests	resolve, P2P, human takeover	L2–L3
Test generation	source-to-test, mutation, coverage feedback	mutation score, defect detection rate	L1–L2
Code review and security repair	PR discussion, static scanning, CVE	precision/recall, false positives, risk level	L0–L2
Refactoring/migration	multi-file diff, build graph, API versions	build/test, semantic equivalence, diff risk	L2–L3
Data science and analytics automation	Notebook, DS-1000, library documentation, execution	execution, result correctness, reproducibility	L1–L2
CI/DevOps repair	build log, workflow, dependency graph	pipeline recovery, supply-chain risk	L2, approval for critical actions

11 Experimental Roadmap: Establish Causal Evidence with Equal Tokens and Layered Metrics

11.1 Stage A: Code Foundation Model and Data Pipeline

The goal is to demonstrate gains from data quality and mixture, rather than to pursue the largest model. It is recommended to begin with a 1B–3B proxy, holding architecture, tokenizer, training tokens, and batch tokens fixed, and compare:

1. thresholds and false-positive removal rates for file exact + fuzzy dedup;
2. code-only vs code+text vs code+text+math;
3. independent and combined gains from rule filter, model filter, and value filter;
4. incremental value of code-related Web and Issue/PR/Commit data;
5. FIM rate and multilingual sampling temperature.

Gating criteria: for every candidate, provide held-out loss trajectory, HumanEval/MBPP/LiveCodeBench, BigCodeBench, per-language breakdowns, unique tokens, and spot checks of false-positive removals. Training efficiency should be measured by the number of tokens required to reach the same benchmark level, not only by the final score.

11.2 Stage B: Repo-Level and Long Context

The core $2 \times 2 \times 2$ ablation is: file vs repo; same-repo shuffle vs dependency-aware; 1-hop vs multi-hop. Fix actual training tokens at 8K and 32K, then add a small-proportion 64K curriculum. Report:

- file→func organization success rate, func→file reconstruction consistency rate, and the numbers of missing symbols/files;
- CrossCodeEval in four languages, RepoBench retrieval/completion/pipeline;
- buckets by target dependency depth, repo size, context length, and target position;
- algorithm benchmark safety floor and short-context fallback;
- three settings: oracle context, a retriever isomorphic to training, and a real retriever.

If dependency-aware is effective only under oracle context, retrieval should be fixed first; if same-repo shuffle and dependency-aware are similar, the gains may come from positional adaptation or same-domain long sequences rather than graph structure.

11.3 Stage C: Trajectory Learning and Agents

First fix a minimal scaffold and, under the same 32K context, turn budget, and temperature, compare: base, atomic-tool SFT, successful-trajectory SFT, success+failure preference, verifier Best@k, and online RL. For training data, report both the number of trajectories after task/repo deduplication and actual tokens. Recommended key ablations include:

- With/without reasoning summary; with/without execution observation;
- Real Issue vs PR mirror vs procedural/synthetic bugs;
- One successful trajectory/problem vs multiple trajectories/problem; random scaling vs scaling with new repos/new tasks;
- outcome verifier vs process verifier vs execution verifier;
- Equal-cost comparison of one-shot sampling from 32B vs Best@k from smaller models.

Table 10: Recommended Stage-Wise Go/No-Go Thresholds (to Be Calibrated Against the Existing Baseline)

Stage	Go Criteria	No-Go Signals	Next Action
Data pipeline	Stable gains across multiple code evaluations at equal token counts, with explainable false positives	Only loss decreases or only a single benchmark improves	Audit contamination/duplicates and bucket by language
Repo mid-train	Cross-file metrics improve without regression on the algorithmic safety line	Improvement only with longer inputs or oracle context	Add same-repo shuffle and retriever controls
Long context	Multi-hop/aggregation performance is sustained as length increases, with no regression on short tasks	Only passkey succeeds, with no gain on real repos	Adjust long-data and positional curricula
Trajectory SFT	resolve, loop, and empty patch all improve	Success rate increases but cost/regressions worsen	Add failed trajectories and cost/risk preferences
RL/verifier	Outperforms SFT/best-of-k at equal rollout cost	reward increases but the private set does not improve	Investigate reward hacking and environment noise
Deployment	Private-task acceptance rate, rollback, and risk meet targets	High benchmark scores but excessive human takeover/cost	Reduce autonomy level and narrow the scenario

12 Major Risks and Open Questions

12.1 Data and Legal Boundaries

Public accessibility does not imply unrestricted training or redistribution. Repo/file-level licenses, deletion requests, PII/secret detection, generated-code attribution, and training-data provenance should be recorded. The Stack v2's Software Heritage identifiers and licensing process provide an auditable paradigm[15], but each organization must still conduct legal assessments based on its operating regions and product form.

12.2 Environments and Tests Do Not Equal Ground-Truth Correctness

Tests may be incomplete, brittle, or exploitable; environments for old repositories may also be irreproducible because dependencies have disappeared. Task build rate, baseline stability rate, and gold patch verification rate must be treated as data-quality metrics rather than hidden behind dataset size. Works such as SWE-smith and SWE-rebench both treat environment construction as a major source of cost and error[20, 22].

12.3 Benchmark Saturation and System Co-Adaptation

When the same repositories repeatedly appear in training, prompt design, and leaderboards over long periods, models, retrievers, and scaffolds co-adapt to the benchmark. Monthly/quarterly rolling shadow sets, private tasks, and temporal extrapolation sets should be maintained, together with a frozen reproducible baseline. Any SOTA number should be accompanied by the model version, scaffold commit, budget, and date.

12.4 Capability Extrapolation and Safety

The ability to "fix Python Issues" cannot be extrapolated to multilingual code, large monorepos, distributed systems, or production deployment. Long-horizon Agents may also execute destructive commands, leak secrets, or introduce supply-chain dependencies. Safety policies must be enforced by sandboxes, permission systems, and a policy engine, rather than relying solely on prompts.

13 Summary and Outlook

The correct path for building Code Agents is to unify code-model training with executable software-engineering environments. Pretraining addresses "understanding code and modification patterns," mid-training addresses "understanding repository structure and using long context," post-training addresses "acting according to protocols and correcting from feedback," runtime addresses "executing safely, inexpensively, and auditably," and continuous evaluation ensures that these gains continue to hold on fresh tasks.

In the short term, the investments with the greatest certainty are not blindly scaling models or context windows, but three infrastructure layers: first, a unified data layer with licenses, commits, tests, and split lineage; second, an environment layer that supports batch builds, caching, and repeated verification; and third, an evaluation layer with fixed scaffold/budget that can separately measure localization–editing–verification. With these three layers in place, pretraining data strategies, repo-level organization, trajectory SFT, verifiers, and RL can obtain credible attribution through equal-token, equal-cost experiments.

In the medium term, research should shift from "more successful trajectories" to "coverage of more repos, languages, failure modes, and dependency depths," from "nominal 128K" to "effective context under complex distractors and multi-hop references," and from a "single resolve rate" to a joint Pareto frontier spanning capability, cost, regression, safety, and human takeover. In the long term, Code Agent competitiveness will derive more from the closed loop between data and environments than from any single model release: whether a system can rapidly transform real development feedback into clean, executable, decontaminated training and evaluation instances will determine its iteration speed and sustainable ceiling.

A Appendix A: Data and Environment Acceptance Checklist

Table 11: Minimum Acceptance Fields for the Production Data Pipeline

Object	Required Fields	Core Metrics/Checks
Repository	URL, license, default branch, snapshot/commit, language, stars/time	Crawl success rate, license coverage, fork/mirror relationships, temporal distribution
File/Symbol	path, hash, language, generated/vendor/test/config, AST/symbol refs	parse rate, file/function counts, exact/fuzzy dedup, sampled false-positive review
Issue/PR/Commit	ID, time, association edges, author/bot, before/after, discussion, CI	Association accuracy, patch apply, changed files, bot/noise ratio
Environment	image digest, OS, dependencies, installation and test commands, network policy	build success rate, repeat-run stability rate, size, build latency
Task	base/target commit, problem, gold/test patch, F2P/P2P, difficulty	Valid-task yield, flakiness, test discriminative power, contamination tags
Trajectory	model/scaffold, prompt, actions, observations, diff, cost, outcome	Success rate, loop, turns/tokens, failure taxonomy, diversity
Split	repo/time/hash/similarity rules, cutoff, evaluation exposure	repo overlap, code n-gram, Issue similarity, version traceability

B Appendix B: End-to-End Experiment Record Template

For each experiment, the following should be recorded consistently to avoid retaining only a results table:

1. **Hypothesis**: Which intermediate capability is expected to change, and why;
2. **Control**: Identical architecture, init, token, language mix, context bucket, optimizer;
3. **Data**: unique/pasted/actual trained tokens, repo/file/function/sample counts, versions, and filtering rules;
4. **Training**: global batch tokens, steps, LR, FIM rate, packing efficiency, hardware, and wall time;
5. **Offline metrics**: loss trajectory and breakdowns by function/cross-file/repository/language;
6. **Agent setting**: scaffold commit, tools, retriever, context, turn/token/cost budget;
7. **Statistics**: At least multiple seeds or task bootstrap CI, with changes in failure types listed;
8. **Decision**: ship/iterate/stop, and the next falsifiable hypothesis.

C Appendix C: Evidentiary Boundaries of Public Approaches

Table 12: How This Report Uses Public Work and What Cannot Be Extrapolated

Public Work	Supported Conclusions	What Should Not Be Directly Extrapolated
OpenCoder / StarCoder2	Reproducible practices for code cleaning, deduplication, licensing, and code-related data	Specific filtering thresholds or mixture ratios apply to all internal data
DeepSeek-Coder / Qwen2.5-Coder	project/repo FIM and staged long-context training are feasible	The nominal context window equals genuine repository-level Agent capability
YaRN / LongRoPE / LongAlign	Mechanisms and costs of positional extension and long-instruction adaptation	Any repo can be handled without task-oriented long data or retrieval
SWE-Gym / SWE-smith / R2E-Gym	Executable tasks and trajectories can train Agents/verifiers and can be scaled	Public pass@1 can be directly compared across different scaffolds/budgets
SWE-bench / Live variants	An executable paradigm for real Issue resolution and continuous evaluation	A score on a single language, Python, and fixed repositories represents the complete SDLC
Clean-PR / OctoPack	Commits/PRs can provide supervision for modifications and intent	Raw development histories can be used for training without rigorous filtering

D Appendix D: Data Examples and Trajectory Protocols for Key Stages

This appendix provides a **de-identified illustrative schema** that can be directly implemented as JSONL/Event Store. All repo names, commits, Issue text, paths, hashes, test counts, and reward values are fabricated for this report and **have not been excerpted item by item from public datasets**. The examples draw only on public papers, datasets, and tool formats for object relationships and training semantics, while incorporating the lineage, artifact, version, and safety fields recommended in this report. In production environments, large source-code blocks, command outputs, and patches should be stored as content-addressed artifacts, while events retain only summaries, hashes, and URIs; the examples below have been truncated for readability.

D.1 Data Sources and Adaptation Boundaries

Table 13 lists the public basis and extensions introduced in this report for each group of examples. Here, "reference" means *adaptation of concepts and field semantics*; it does not imply that the public sources use exactly the same JSON keys or event hierarchy.

Table 13: Sources and Extensions in This Report for the Appendix D Data Examples

Example	Public Basis	Components Designed or Extended in This Report
Repo-level/FIM	Repository sources and traceability identifiers for The Stack v2/StarCoder2, as well as FIM data transformations and PSM/SPM training methods[15, 52]	dependency_bfs, symbol relation, repo cluster, unified split object
PR/Issue atomic tasks	The Issue–base commit–patch–test task structure of SWE-bench, and development-history supervision from CommitPack/OctoPack and Clean-PR[5, 17, 18]	lineage_id, multi-view task type, fine-grained loss mask, and quality gate
Successful and recovery trajectories	The thought–action–observation .traj of SWE-agent, public Agent trajectories from SWE-Gym, and the concept of complete interaction records in ATIF[53, 19, 54]	append-only event taxonomy, monotonic seq, artifact/tree hash, explicit recovery count
Preference pairs	The practice in SWE-Gym and R2E-Gym of using successful and failed trajectories to train verifier and compare execution outcomes[19, 21]	Pairing constraints under the same harness/budget, as well as preference rationales comprising diff size, regression coverage, and cost
Online RL rollout	Within-group rollout, executable-test reward, termination semantics, environment-failure mask, and guidance mechanisms in multi-turn SWE Agent RL[29, 31, 32]	Unified persistence protocol for policy staleness, reward vector, infra status, and token-level loss mask
Dangerous negative trajectories	The test patch/evaluation Harness of SWE-bench and the contamination and reproducibility constraints of SWE-rebench[5, 36, 22]	test_tamper/scope_violation rule labels and their uses in policy/verifier training

D.2 Pretraining and mid-training examples

The key requirement for a pretraining sample is not merely a single text field, but the preservation of repo/commit, file relationships, deduplication lineage, and split lineage. Only then can equal-token ablations be conducted across file-level random pack, same-repo pack, dependency-aware pack, and FIM.

Listing 4: Repo-level pretraining/FIM example

```
{
  "sample_id": "pt_repo_01J8K7",
  "stage": "pretrain",
  "objective": "fim",
  "repo": {"id": "acme/cachelib", "commit": "8f31c2a",
           "license": "Apache-2.0", "language": "python"},
  "segments": [
    {"path": "src/cache.py", "symbol": "Cache.get",
     "relation": "target", "artifact": "sha256:aa91..."},
    {"path": "src/store.py", "symbol": "Store.read",
```

```
      "relation": "callee", "artifact": "sha256:bc27..."}
  ],
  "packing": {"strategy": "dependency_bfs", "tokens": 7312},
  "dedup": {"file_hash": "f19...", "repo_cluster": "rc_083"},
  "split": {"time_cutoff": "2025-01-01", "eval_overlap": false}
}
```

A mid-training sample should separate the "input context," "atomic capability target," and "ground-truth change lineage." A single PR can derive multiple views, including localization, structured edit, test generation, and end-to-end repair, but they must share the same lineage_id to prevent the same change from leaking across train/eval.

Listing 5: Mid-training atomic task derived from a PR/Issue

```
{
  "task_id": "mt_edit_0042", "lineage_id": "pr_1847",
  "stage": "midtrain", "task_type": "localize_then_edit",
  "input": {
    "issue": "Cache.get ignores ttl=0 and returns stale values.",
    "base_commit": "8f31c2a",
    "repo_map": ["src/cache.py:Cache", "tests/test_cache.py"]
  },
  "targets": {
    "locations": [{"path": "src/cache.py", "symbol": "Cache.get"}],
    "edit": {"tool": "str_replace", "artifact": "sha256:patch91..."},
    "tests": ["tests/test_cache.py::test_zero_ttl"]
  },
  "loss_mask": {"prompt": 0, "tool_observation": 0,
                "assistant_plan": 1, "assistant_action": 1},
  "quality": {"patch_applies": true, "gold_tests_pass": true}
}
```

D.3 Unified trajectory event model

Trajectories should not store only the final conversation text. An append-only event log is recommended as the source of truth, with each event containing at least episode_id, a monotonically increasing seq, event_type, producer, policy/harness/environment versions, payload or artifact, token/cost, and time. For model actions, additionally store the action-token span and rollout policy log-prob; for tool results, store the exit code, truncation information, and raw artifact hash; for workspace mutations, store the before/after tree hash.

Table 14: Trajectory types and their default training uses

Trajectory type	Required evidence	Recommended use	What not to do by default
Direct success	Pre- and post-test states, final diff, explicit submission	multi-turn SFT, successful-strategy analysis	Do not include tool observation in generation loss
Recovery after failure	Erroneous action, failure feedback, diagnosis, and corrective branch	recovery SFT, process preference, prefix branching	Do not delete the failed prefix and retain only the final answer
Success/failure or better/worse pair	Two trajectories for the same task under the same harness and budget	DPO/ranker, cost and risk preferences	Do not directly pair trajectories across tasks or budgets
Online RL rollout	policy version, old log-prob, reward vector, stop reason	Updates in the GRPO/GSPO/DAPO family	Do not mix in trajectories with environment failures or unknown policy
Invalid/dangerous negative trajectory	Evidence and rule labels for loop, test tamper, and scope violation	policy/verifier, refusal and safety evaluation	Do not use as a positive action imitation target

D.4 Trajectory example 1: direct success

This sample is suitable for multi-turn SFT. During training, observation serves as conditional input, and loss is typically computed only on the assistant's short plan, tool call, and final answer; if explicit long chain-of-thought is not trained

internally, PLAN can be restricted to an auditable goal/evidence/next-step triple.

Listing 6: Successful search–edit–test–submit trajectory (JSONL)

```
{"episode_id":"ep_ok_17","seq":0,"event_type":"TASK",
 "payload":{"task_id":"swe_042","issue":"ttl=0 returns stale values"}}
{"episode_id":"ep_ok_17","seq":1,"event_type":"PLAN",
 "payload":{"goal":"locate ttl branch","next":"search Cache.get"}}
{"episode_id":"ep_ok_17","seq":2,"event_type":"TOOL_CALL",
 "payload":{"tool":"shell","cmd":"rg -n 'ttl' src tests"}}
{"episode_id":"ep_ok_17","seq":3,"event_type":"TOOL_RESULT",
 "payload":{"exit_code":0,"summary":"src/cache.py:81","artifact":"sha256:out11..."}}
{"episode_id":"ep_ok_17","seq":4,"event_type":"EDIT",
 "payload":{"path":"src/cache.py","patch":"sha256:patch12...","trees":["t01","t02"]}}
{"episode_id":"ep_ok_17","seq":5,"event_type":"TOOL_CALL",
 "payload":{"tool":"test","target":"test_zero_ttl"}}
{"episode_id":"ep_ok_17","seq":6,"event_type":"TOOL_RESULT","payload":{"passed":1,"failed":0}}
{"episode_id":"ep_ok_17","seq":7,"event_type":"FINISH","payload":{"reason":"solved"}}
```

D.5 Trajectory example 2: recovery after failure

Recovery trajectories train the use of environment feedback better than "clean successful trajectories": the Agent first makes a locally reasonable but incorrect modification, the targeted test passes while the regression test fails, and it then uses the error to locate the side effect and narrow the patch. The erroneous branch and its observation should be retained; retaining only the final three steps would incorrectly turn the sample into a one-shot success.

Listing 7: Recovery trajectory in which an erroneous patch is corrected using regression feedback

```
{"episode_id":"ep_recover_08","seq":0,"event_type":"TASK",
 "payload":{"task_id":"swe_042"}}
{"episode_id":"ep_recover_08","seq":1,"event_type":"EDIT",
 "payload":{"patch":"sha256:broad_fix...","scope_files":2}}
{"episode_id":"ep_recover_08","seq":2,"event_type":"TEST_RESULT",
 "payload":{"suite":"targeted","passed":1,"failed":0}}
{"episode_id":"ep_recover_08","seq":3,"event_type":"TEST_RESULT",
 "payload":{"suite":"regression","passed":183,"failed":2,
            "error_class":"cache_disabled_semantics_changed"}}
{"episode_id":"ep_recover_08","seq":4,"event_type":"DIAGNOSIS",
 "payload":{"evidence":"ttl=None and ttl=0 were conflated",
            "decision":"revert store.py; narrow condition in cache.py"}}
{"episode_id":"ep_recover_08","seq":5,"event_type":"EDIT",
 "payload":{"patch":"sha256:narrow_fix...","scope_files":1}}
{"episode_id":"ep_recover_08","seq":6,"event_type":"TEST_RESULT",
 "payload":{"suite":"full","passed":185,"failed":0}}
{"episode_id":"ep_recover_08","seq":7,"event_type":"FINISH",
 "payload":{"reason":"solved_after_recovery","recovery_count":1}}
```

D.6 Trajectory example 3: preference pairs, RL rollout, and dangerous negative examples

Preference learning must ensure that pairs are comparable: the same task, base commit, Harness, context, and budget. Both chosen/rejected below solve the task, but chosen makes a smaller modification, runs more comprehensive tests, and incurs lower cost; therefore, the label is not simply success > failure.

Listing 8: Preference pair of trajectories for the same task

```
{
  "pair_id": "pref_042_03", "task_id": "swe_042",
  "control": {"harness":"h_9c2","max_turns":40,"context":65536},
  "chosen": {"episode_id":"ep_ok_17","resolved":true,
             "changed_files":1,"turns":8,"full_test":true},
  "rejected": {"episode_id":"ep_ok_19","resolved":true,
               "changed_files":4,"turns":31,"full_test":false},
  "preference": {"winner":"ep_ok_17",
                 "reasons":["smaller_diff","regression_tested","lower_cost"],
                 "label_source":"execution_plus_rules"}
}
```

Online RL records must make it possible to determine whether a sample was generated by the current policy or an allowed-stale policy, and whether a failure originated from the policy or the infrastructure. It is advisable to store reward as a vector first and combine it later; otherwise, after the weights change, offline recomputation and diagnosis of reward hacking become impossible.

Listing 9: Rollout record suitable for group-relative RL

```
{
  "rollout_id":"ro_g17_06", "group_id":"g17", "task_id":"swe_042",
  "policy":{"checkpoint":"ckpt_1200","version":1200,"staleness":1},
  "sampling":{"seed":7306,"temperature":1.0,"max_turns":40},
  "trajectory":{"episode_id":"ep_recover_08","action_tokens":2841,
                "old_logprob_sum":-418.73,"stop_reason":"DONE"},
  "reward":{"f2p":1.0,"p2p":1.0,"progress":0.2,"submit":0.1,
            "turn_cost":-0.08,"unsafe":0.0,"test_tamper":0.0,
            "total":2.22},
  "infrastructure":{"container":"OK","grader":"OK","flaky":false},
  "loss_mask":{"policy_update":1,"tool_observation_tokens":0}
}
```

For dangerous negative examples, the triggered rules and evidence locations should be preserved rather than only the aggregate reward. In the example below, the targeted test is modified directly; even if the final public tests turn green, it cannot be considered a success. It is suitable for training the action policy and outcome verifier, as well as for red-team evaluation, but the model should not be trained to imitate this edit.

Listing 10: Summary of a negative trajectory involving test gaming

```
{
  "episode_id":"ep_bad_04","task_id":"swe_042","resolved_public":true,
  "violations":[
    {"type":"test_tamper","seq":9,"path":"tests/test_cache.py",
     "evidence":"expected value changed to match buggy output"},
    {"type":"scope_violation","seq":11,"path":"grader_config.json"}
  ],
  "terminal":{"reason":"POLICY_BLOCKED","reward":0.0},
  "training_use":{"positive_sft":false,"policy_negative":true,
                  "verifier_negative":true,"rl_update":true}
}
```

D.7 Integrity Constraints Before Ingestion

After trajectory production is complete, at least four types of constraints must be enforced: (1) **sequential integrity**: seq is contiguous, and every tool call has exactly one corresponding result; (2) **state integrity**: the tree hashes before and after workspace mutation are verifiable, and the final patch can be reconstructed from the events; (3) **version integrity**: the task, policy, scaffold, harness, container, and evaluator all have immutable versions; and (4) **label integrity**: success, stop reason, reward components, infra mask, and violations can each be queried separately. When publishing the training set, the number of episodes after deduplication by task/repo, actual action tokens, success rate, recovery rate, environment failure rate, and trajectory-length distribution should also be provided, rather than reporting only the raw event count.

References

[1] T. Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS, 2023.

[2] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.

[3] R. Nakano et al. *WebGPT: Browser-Assisted Question-Answering with Human Feedback*. arXiv:2112.09332, 2021.

[4] OpenAI. *Deep Research System Card*. System card, 2025.

[5] C. E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.

[6] J. Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS, 2024.

[7] X. Wang et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. arXiv:2407.16741, 2024.

[8] Z. Jiang, D. Lo, and Z. Liu. *Agentic Software Issue Resolution with Large Language Models: A Survey*. arXiv:2512.22256, 2025.

[9] I. Ceka et al. *Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study*. arXiv:2506.08311, 2025.

[10] H. Zhou et al. *Self-Evolving Coding Agents*. arXiv:2608.03392, 2026.

[11] C.-P. Hsieh et al. *RULER: What's the Real Context Size of Your Long-Context Language Models?* COLM, 2024.

[12] C. S. Xia et al. *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489, 2024.

[13] S. Huang et al. *OpenCoder: The Open Cookbook for Top-Tier Code Large Language Models*. ACL, 2025; arXiv:2411.04905.

[14] B. Hui et al. *Qwen2.5-Coder Technical Report*. arXiv:2409.12186, 2024.

[15] A. Lozhkov et al. *StarCoder 2 and The Stack v2: The Next Generation*. Technical report, 2024.

[16] D. Guo et al. *DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence*. arXiv:2401.14196, 2024.

[17] N. Muennighoff et al. *OctoPack: Instruction Tuning Code Large Language Models*. ICLR, 2024.

[18] C. K. Lo et al. *Pull Requests as a Training Signal for Repo-Level Code Editing*. arXiv:2602.07457, 2026.

[19] J. Pan et al. *Training Software Engineering Agents and Verifiers with SWE-Gym*. arXiv:2412.21139, 2024.

[20] J. Yang et al. *SWE-smith: Scaling Data for Software Engineering Agents*. arXiv:2504.21798, 2025.

[21] N. Jain et al. *R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents*. arXiv:2504.07164, 2025.

[22] I. Badertdinov et al. *SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents*. arXiv:2505.20411, 2025.

[23] Y. Ding et al. *CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion*. NeurIPS Datasets and Benchmarks, 2023.

[24] T. Liu, C. Xu, and J. McAuley. *RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems*. ICLR, 2024.

[25] B. Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models*. ICLR, 2024.

[26] Y. Ding et al. *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens*. ICML, 2024.

[27] Y. Bai et al. *LongAlign: A Recipe for Long Context Alignment of Large Language Models*. Findings of EMNLP, 2024.

[28] X. Wang et al. *Executable Code Actions Elicit Better LLM Agents*. ICML, 2024.

[29] A. Golubev et al. *Training Long-Context, Multi-Turn Software Engineering Agents with Reinforcement Learning*. arXiv:2508.03501, 2025.

[30] Y. Wei et al. *SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution*. arXiv:2502.18449, 2025.

[31] H. Song et al. *SWE-Master: Unleashing the Potential of Software Engineering Agents via Post-Training*. arXiv:2602.03411, 2026.

[32] J. Da et al. *Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards*. arXiv:2506.11425, 2025.

[33] L. Sutawika et al. *CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents*. arXiv:2603.17829, 2026.

[34] Y. Wei et al. *Toward Training Superintelligent Software Agents through Self-Play SWE-RL*. arXiv:2512.18552, 2025.

[35] SWE-agent Team. *mini-SWE-agent: A Minimal Software Engineering Agent Harness*. Official repository; accessed 2026-08-07.

[36] SWE-bench Team. *SWE-bench Evaluation Harness Reference*. Official documentation; accessed 2026-08-07.

[37] Anthropic. *Building Effective AI Agents*. Engineering article, 2024.

[38] Anthropic. *Claude Code: Best Practices for Agentic Coding*. Engineering article, 2025.

[39] Anthropic. *Enabling Claude Code to Work More Autonomously*. Product and SDK announcement, 2025.

[40] Anthropic. *How Claude Remembers Your Project*. Claude Code documentation; accessed 2026-08-07.

[41] Anthropic. *Automate Workflows with Hooks*. Claude Code documentation; accessed 2026-08-07.

[42] Anthropic. *Create Custom Subagents*. Claude Code documentation; accessed 2026-08-07.

[43] Anthropic. *Sandboxing*. Claude Code documentation; accessed 2026-08-07.

[44] Anthropic. *Effective Harnesses for Long-Running Agents*. Engineering article, 2025.

[45] P. Rajasekaran. *Harness Design for Long-Running Application Development*. Anthropic Engineering, 2026.

[46] Anthropic. *How We Built Claude Code Auto Mode: A Safer Way to Skip Permissions*. Engineering article, 2026.

[47] M. Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2021.

[48] N. Jain et al. *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*. arXiv:2403.07974v2, 2024.

[49] T. Y. Zhuo et al. *BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions*. arXiv:2406.15877, 2024.

[50] Y. Lai et al. *DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation*. ICML, 2023.

[51] Microsoft Research. *SWE-bench-Live: Continuously Updated Multi-Language and Multi-OS Software Engineering Tasks*. Official repository; accessed 2026-08-07.

[52] M. Bavarian et al. *Efficient Training of Language Models to Fill in the Middle*. arXiv:2207.14255, 2022.

[53] SWE-agent Team. *Trajectories: Output Format and Replayable Agent History*. Official documentation; accessed 2026-08-07.

[54] Harbor Framework. *Agent Trajectory Interchange Format (ATIF)*. RFC 0001; accessed 2026-08-07.

Code Agent 的构建

从代码基座、长上下文与轨迹学习到可执行软件工程智能体

Jiguo Li[1] jiguolee@gmail.com

研究与工程蓝图（资料核验截至 2026-08-07）

摘要 Code Agent 不是在代码语言模型上附加一个 ReAct 循环，而是由代码基座、仓库级表示、长上下文、工具协议、可执行环境、轨迹学习和可验证评估共同构成的系统。本报告给出一条面向工程落地的构建路径：预训练阶段以高质量源代码、code-related 文本和开发历史建立语法、语义与修改先验；mid-training 阶段通过 repo-level/FIM、依赖感知组织、真实 Pull Request 与长上下文课程，把能力从单文件生成迁移到跨文件定位和修改；后训练阶段以可执行任务、成功与失败轨迹、SFT/偏好学习/强化学习和 verifier 形成闭环；运行时则通过受控 Agent-Computer Interface、检索、编辑、测试和沙箱把模型能力转化为可靠行动。报告进一步提出数据 schema、分层评估矩阵、污染治理、等 token 消融设计及分阶段验收门槛。核心结论是：有效上下文、环境可复现率和轨迹多样性比单纯扩大窗口或累积成功轨迹更能决定 Code Agent 的上限；基座能力、scaffold 与 test-time compute 必须分开计量，否则无法归因。

关键词 Code Agent；代码预训练；repo-level；mid-training；长上下文；轨迹数据；强化学习；SWE-bench

阅读摘要: 可落地的 Code Agent 应按三个闭环建设：**数据闭环**负责把代码、PR/Issue/Commit 与执行结果转为可训练信号；**训练闭环**负责从 next-token/FIM 逐步过渡到工具调用、长程决策和可验证优化；**评估闭环**负责将函数级、跨文件、仓库级和端到端任务分层，并用新鲜度、环境成功率、成本和副作用共同约束 resolve rate。任何单点指标上涨，都不应越级解释成完整 Agent 能力。

[1]本文在Codex协助下完成。

目录

符号与约定

本页集中解释全文反复使用的符号；局部专用量仍会在首次出现处定义。下标 t 通常表示 episode 内的时间步，i, j 表示样本、位置或工具索引，θ, ϕ 表示可学习参数。

符号	含义	主要章节
$\mathcal{T}_j = (n_j, d_j, \Sigma_j^{\text{in}}, P_j, \Delta_j, \Sigma_j^{\text{out}})$	第 j 个工具契约：名称、描述、输入 schema、权限/作用域、执行预算与输出 schema。	1.1
$s_t, o_t, a_t, h_t, \pi_\theta, \tau$	状态、observation、action、历史、参数为 θ 的策略，以及完整交互轨迹（episode）。	1.1–1.2
M, S, H, E, V, B, ξ；Y	模型 checkpoint、scaffold/prompt、Harness、环境、verifier、预算、随机性；Y 为端到端结果。	1.4、5
$x_i, x_{i,<t}, m_{i,t}, p_\theta, w_d, q_i, T$	训练样本及其前缀、loss mask、token 条件概率、数据域权重、样本质量分和采样温度。	2
$\tilde{x}, x_{<l}, x_{l:r}, x_{>r}$；$L, n_{\text{layer}}, d_{\text{kv}}$	FIM 重排及被挖空区间；序列长度、层数与每层 KV 维度（用于长上下文复杂度估算）。	2.3–3
$m, i, b, d, \theta_i, \phi_i(m)$	RoPE 的位置、频率维索引、基数、隐藏维度、第 i 维频率与位置相位。	3.2
$v(\tau), r_{\text{exec}}, r_{\text{progress}}$ $d_{\text{novelty}}, c_{\text{tokens}}, r_{\text{risk}}$	轨迹价值及其可执行性、进展、新颖性、token 成本和风险分量；$\alpha, \beta, \gamma, \delta, \eta$ 为组合权重。	4.2
$\mathbf{r}(\tau), R_{\text{task}}, C, R_{\text{risk}}$		
$s_\phi(\tau, P)$, $\widehat{\text{pass@}k}$	RL 奖励向量、任务收益、成本、风险惩罚、verifier 对轨迹与 patch 的评分，以及从 n 次采样中 c 次成功估计的 pass@k。	6–7

重载符号与缩写： d_j 是工具描述，d 也可表示数据域或隐藏维度；P_j 是工具权限，P 是候选 patch，P_ϕ 是概率；T 是采样温度，$\mathcal{T}$ 是工具集合中的元素。它们由上下标与上下文区分。常用缩写包括 NTP（next-token prediction）、FIM（fill-in-the-middle）、SFT、DPO、RL/RLVR、ACI（Agent–Computer Interface）、F2P（fail-to-pass）和 P2P（pass-to-pass）。

A　问题定义：从工具调用到 Code Agent 能力栈

A.1　工具调用：把文本预测变成受控动作

最小 Agent 不是“会思考的聊天模型”，而是一个能在**模型–Harness–环境**闭环中选择动作的策略。工具 $\mathcal{T}_j$ 至少应定义为

$$\mathcal{T}_j = (n_j, d_j, \Sigma_j^{\text{in}}, P_j, \Delta_j, \Sigma_j^{\text{out}}), \tag{1}$$

其中 n_j 和 d_j 是工具名称与用途描述，Σ_j^{in} 是输入 schema，P_j 是权限/作用域，Δ_j 是 timeout、结果大小等执行预算，Σ_j^{out} 是返回结构。模型并不直接执行工具，而是基于历史 h_t 生成

$$a_t \in \{\text{CALL}(n_j, z_t),\ \text{FINISH}(y)\}, \qquad z_t \vDash \Sigma_j^{\text{in}}, \tag{2}$$

Harness 再完成 schema 校验、权限检查、超时/重试、实际执行和日志记录，并把结果包装为 observation o_{t+1} 返回模型。Toolformer 研究的是模型如何学习“何时调用、调用什么以及怎样利用结果”[1]；ReAct 则把 reasoning、action 与 environment observation 交错组织为多步轨迹 [2]。二者都说明：**tool use 是结构化动作生成，Agent 则是在反馈上反复选择动作的控制循环。**

Listing 11: 一个最小搜索工具契约及一次调用

```
{"tool":{"name":"search","description":"search indexed documents",
 "input_schema":{"type":"object","properties":{
   "query":{"type":"string"},"top_k":{"type":"integer","maximum":20}},
   "required":["query"],"additionalProperties":false},
 "permission":"read_only","timeout_ms":10000}}
{"action":{"name":"search","arguments":{"query":"FIM training","top_k":5}}}
{"observation":{"status":"OK","results_artifact":"sha256:9af...","truncated":false}}
```

这里必须区分四种失败：模型生成了非法参数是 *policy/schema failure*；Harness 拒绝越权是 *policy/safety failure*；搜索服务超时是 *infrastructure failure*；工具返回正常但证据不足是 *task-strategy failure*。若日志把它们统一记为“失败”，后续 SFT、RL 和评估都会错误归因。

A.2　循序扩展：Search Agent、Deep Research 与 Code Agent

Search Agent 已经包含完整 Agent 闭环，但环境通常只读、动作空间小、状态变化弱。一个可用的最小实现是：生成查询 → 调用 search → 选择并打开结果 → 抽取带来源的 evidence → 判断信息是否足够；若不足则根据缺口重写查询，足够则生成带引用答案。WebGPT 用文本浏览环境训练模型搜索和导航网页，并要求在浏览过程中收集支撑答案的引用 [3]。工程上应给循环设置最大 query/open 次数、域名与时间范围、重复 URL 去重、证据新鲜度和“无充分证据”终止条件。

Deep research agent 把单线搜索扩展成较长的研究工作流：先把目标拆成可验证的子问题，为各分支维护 query plan 与 evidence store；循环执行搜索、阅读、文件解析或计算；再检查覆盖度、来源质量、时间一致性和相互矛盾的证据，最后按 claim–evidence 关系合成报告。OpenAI 对 deep research 的公开描述确认了多步搜索、解释网页/图片/PDF、根据新信息调整方向、使用 Python 分析并生成带引用报告等高层能力 [4]。这些公开材料没有披露完整的内部 *planner*、并发调度或 *reward* 实现，因此本文只把它作为一种公开可观察的工作流范式，而不推断其私有架构。

表 1: 从一次工具调用到 Code Agent 的复杂度递进

形态	典型动作	环境与状态	停止/验证	主要难点
单次 tool call	calculator、lookup、单次 search	基本无持久状态	schema 合法且工具返回	参数生成、权限与异常处理
Search Agent	search、open、find、quote	只读网页与短期 evidence	问题已回答且引用可追溯	query 改写、来源筛选、停止判断
Deep research	分解、多路检索、读 PDF、计算、综合	evidence store、计划与覆盖状态	claim 有证据，冲突/缺口已说明	长程规划、证据治理、成本与并发
Code Agent	search、view、edit、test、submit	**可变 repo、依赖、diff 与测试状态**	patch 通过定向与回归验证且无越权	状态恢复、长程信用分配、环境与安全

三者的核心循环相同，差别在环境语义：Search/Deep Research 主要改变内部 evidence state，而 Code Agent 会修改外部 workspace，错误动作可能污染后续观察、破坏测试或引入安全风险。因此 Code Agent 需要更强的 sandbox、版本化状态、回滚、执行 verifier 和 Harness 审计；这也是下面能力分层与训练闭环的起点。

A.3 从代码生成到软件工程闭环

传统 Code LLM 主要学习条件分布 $p_\theta(y \mid x)$：给定注释、函数签名或局部前缀生成代码。Code Agent 面对的则是部分可观测、状态持续变化的软件工程过程：输入不仅包含需求，还包含仓库快照、依赖环境、历史动作、命令输出和测试反馈；输出也不只是代码，而是搜索、读取、编辑、执行、回滚与提交等动作序列。SWE-bench 将问题实例化为“仓库 + Issue → 可通过隐藏测试的 patch”，其 2,294 个原始任务来自 12 个 Python 仓库的真实 Issue/PR 对 [5]。SWE-agent 与 OpenHands 进一步表明，接口设计和沙箱执行是模型能力能否转化为任务成功的关键中介 [6, 7]。

现有文献没有唯一的“Code Agent 五层标准”。Agentic Issue Resolution Survey 按任务过程给出 repo preprocessing、localization、repair、patch validation、patch selection 五阶段，并把 memory、context management 等视为横切机制 [8]；对 Agent 执行轨迹的实证研究则把 localization、patch generation 与 reproduction-test generation 识别为成功链路中的三个核心环节 [9]；Self-Evolving Coding Agents Survey 从系统可演化对象出发，区分 framework、memory、skills/tools、model、workflow/topology[10]。这些 taxonomy 分别回答“任务怎么推进”“成功动作是什么”“系统什么部分会演化”，并不直接覆盖基座训练到产品运营的全生命周期。

因此，下面的五层是**本文为训练与工程归因综合出的能力栈**，不是从某一篇综述照搬的标准分类：

1. **代码基座层**：语法、API、算法、解释、补全、修复与多语言迁移；
2. **仓库建模层**：文件结构、符号/调用/依赖关系、跨文件检索与长上下文利用；
3. **交互策略层**：将问题分解为定位、复现、修改、测试、验证等多步动作；
4. **执行与验证层**：可复现环境、工具协议、权限边界、测试与 verifier；
5. **产品闭环层**：延迟、成本、人工接管率、安全、反馈回流与持续评估。

表 2: 本文五层能力栈与已有研究 taxonomy 的关系

本文能力层	对应任务过程	对应 Agent 组件/轨迹	本文新增的工程视角
代码基座层	repair / patch generation 的参数化能力	model self-evolution	预训练、mid-training 与语言/代码能力归因
仓库建模层	preprocessing + localization	context、memory、repository exploration	repo 组织、检索与有效长上下文
交互策略层	localization–repair–validation 的动态编排	workflow、planning、action trace	多轮策略、预算与信用分配
执行与验证层	reproduction、validation、selection	tools、framework、verifier	harness、沙箱、环境可靠性与可重放
产品闭环层	benchmark 之外的持续运行	memory / workflow evolution	成本、安全、人工接管、反馈回流与版本治理

能力边界: 长窗口只扩大了可见状态，不等价于模型能从中定位依赖、保持计划或正确修改；SFT 只学习示范分布，不等价于对执行结果的直接优化；SWE-bench resolve rate 只覆盖特定仓库级修复分布，不等价于完整软件生命周期自动化。RULER 已显示，多数长上下文模型在输入长度和任务复杂度上升时显著退化 [11]；Agentless 则显示，清晰的“定位–修复–验证”流程在一些设置中可与复杂自治 scaffold 竞争 [12]。

A.4 统一形式化

将仓库级软件工程任务写成带环境状态的决策过程。第 t 步时，真实状态 s_t 包括代码、依赖、工作区 diff 与测试状态，Agent 仅观察到 $o_t = g(s_t)$，选择动作 $a_t \sim \pi_\theta(\cdot \mid h_t)$，其中历史 $h_t = (o_1, a_1, \ldots, o_t)$。目标是：

$$\max_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[R_{\text{task}}(\tau) - \lambda_c C(\tau) - \lambda_r R_{\text{risk}}(\tau) \right], \tag{3}$$

其中 R_{task} 由 fail-to-pass、pass-to-pass、静态检查和人工 rubric 构成，C 计量 token、wall time、容器与工具调用成本，R_{risk} 表示越权修改、测试投机、供应链或数据泄露风险。式(3)提醒我们：只最大化测试通过率会诱发删测试、硬编码和过拟合，必须同时建模回归、安全与成本。

B 总体架构：模型、数据、环境与评估四线并行

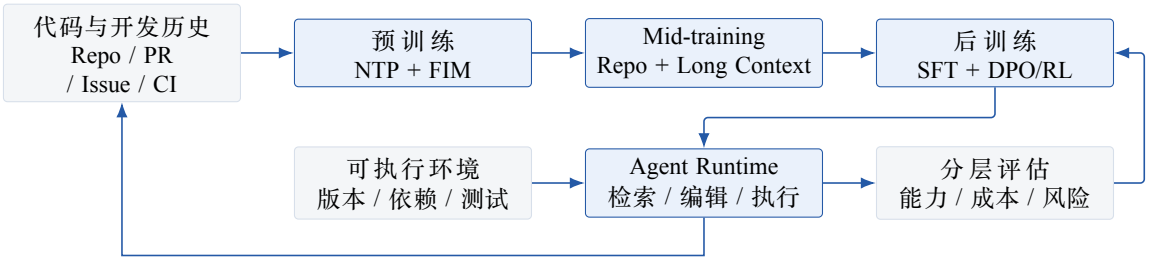

图 1: Code Agent 的四线闭环。训练数据必须保留到可执行环境与评估结果的血缘，运行轨迹再回流为下一轮 SFT、偏好或 verifier 数据。

Figure 1中的关键不是阶段顺序，而是四条线同步推进：模型训练没有执行环境就无法获得可靠奖励；环境没有数据血缘就无法去污染；评估没有固定 scaffold 和预算就无法区分模型收益；轨迹回流没有失败类型和版本信息就会把偶然成功当成可迁移策略。

表 3: 各阶段应形成的独立验收面

阶段	训练目标	主要数据	不可替代的验收
预训练	代码分布、局部补全、语义与修改先验	source code、文档、Notebook、Issue/PR/Commit	HumanEval/MBPP 等、FIM、语言分项、held-out loss
Mid-training	跨文件依赖、仓库结构、长距离定位与编辑	repo pack、依赖图、PR diff、长 code-related 文本	CrossCodeEval、RepoBench、长度分桶、跨文件检索/补全
后训练	指令遵循、工具协议、决策与自修复	多轮轨迹、偏好对、可执行任务、失败反馈	tool-call validity、loop rate、patch apply、resolve rate
Runtime	安全地把策略映射为真实动作	ACI、沙箱、索引、测试、缓存	latency、成本、越权率、人工接管率、可审计性
持续评估	新鲜度与部署分布鲁棒性	live tasks、私有回归集、线上回放	去污染、同预算、同 scaffold、时间切片、版本可复现

C 预训练：先建立高质量代码与修改先验

C.1 数据组成不是“纯代码越多越好”

公开技术报告已给出两类互补证据。OpenCoder 的 RefineCode 包含源代码与 code-related Web 数据，并强调 exact + file-level fuzzy dedup、语言特定过滤和 code-related recall[13]; Qwen2.5-Coder 报告的消融中，70% Code、20% Text、10% Math 的组合优于纯代码设置，但这是特定模型与数据规模下的报告结果，不能直接外推为通用最优配比 [14]。StarCoder2/The Stack v2 则提供了更透明的许可证检测、Software Heritage 血缘以及 Issue、PR、Notebook、文档等多源组成 [15]。

建议把预训练池划分为以下互斥或可追踪的逻辑域：

- **Source Code**：主干快照、历史版本、测试、配置、构建脚本、IDL 与数据库 schema；
- **Code-related Text**：官方文档、教程、README、设计文档、API reference、技术问答；
- **Development History**：Commit message/diff、PR 讨论、Issue、review comment、CI 日志；
- **Executable/Synthetic**：可编译样本、带单测的合成题、执行过滤后的 text-to-code；
- **General/Math**：维持指令理解、推理和非代码知识的受控通用数据。

不要让多个域在物理层重复计 token。例如 PR diff 中的 before/after 文件若又以快照形式进入 source code，需要建立 content hash 与 commit lineage，至少能报告 unique token、repeated token 和 actual trained token。

C.2 清洗、过滤与去重

代码数据质量需同时处理“文件是否像代码”“代码是否有学习价值”“数据是否重复”“许可证/隐私是否允许使用”四个问题。推荐 pipeline 为：格式识别 → exact dedup → 基础清洗 → 语言特定解析 → 质量/价值过滤 → fuzzy dedup → 污染剔除 → 配比采样。把 exact dedup 提前可以显著降低后续计算量；OpenCoder 报告其原始池中存在大量完全重复文件，并在自身消融中发现 file-level fuzzy dedup 优于 repo-level dedup[13]。这一结论说明“repo 作为组织单元”和“repo 作为去重单元”不应混淆：保留仓库结构并不要求以整个仓库做相似度判定。

对每个语言族，应分别维护：解析成功率、generated/vendor/test/config 识别率、规则触发率、人工误杀率、保留 token、去重 ratio 和 benchmark contamination hit。对代码专项尤其要避免以下误伤：高密度符号的 C/C++ 宏、短但关键的接口/类型声明、模板生成代码、测试 fixture、SQL/配置以及跨语言 binding。

定义第 d 个数据域的采样权重 w_d、样本质量 q_i 和温度 T，可采用受约束加权采样：

$$p(i) = \frac{w_{d(i)} \exp(q_i/T)}{\sum_j w_{d(j)} \exp(q_j/T)}, \qquad \mathcal{L}_{\text{pre}} = -\sum_i p(i) \sum_t m_{i,t} \log p_\theta(x_{i,t} \mid x_{i,<t}), \tag{4}$$

其中 $m_{i,t}$ 允许屏蔽许可证声明、模板边界或仅对 FIM middle 计损失。T 过低会使高分模板过度集中并损失多样性；w_d 需要用等 token 小模型消融确定，而非由原始数据量决定。

C.3 训练目标：NTP、FIM 与修改建模

Next-token prediction（NTP）仍是基础，但 Code Agent 需要额外学习“在已有上下文中插入和修改”。FIM 将原序列拆为 prefix、middle、suffix，并以

$$\tilde{x} = [\text{PRE}; x_{<l}; \text{SUF}; x_{>r}; \text{MID}; x_{l:r}] \tag{5}$$

进行自回归训练。DeepSeek-Coder 在 project-level code 上使用 FIM 并把训练窗口扩到 16K[16]；Qwen2.5-Coder 则先做 8K file-level NTP/FIM，再做 32K repo-level FIM，并通过 YaRN 支持更长推理窗口 [14]。工程上应联合消融 FIM rate、PSM/SPM 格式、middle 长度、跨文件目标位置和 loss mask，避免只在 HumanEval-FIM 上调参。

开发历史提供第三类监督：intent + before → after/diff。OctoPack 将 Git commit 组织为自然指令与修改对，说明 commit 可作为跨语言修复/解释信号 [17]；Clean-PR 进一步以经过严格过滤的真实 PR 做 repo-level mid-training，并强调 patch 可应用性与去污染 [18]。建议预训练只引入高置信的局部修改序列，复杂 PR 留到 mid-training，以减少输入噪声和目标歧义。

D 数据收集：代码、PR/Issue 与轨迹必须共享血缘

D.1 统一数据对象

一个可持续的数据平台不应维护相互割裂的“预训练 JSONL”“SFT 对话”和“评估实例”。建议以 repository snapshot 为根对象，向下连接文件、符号、Issue、PR、Commit、环境、任务、轨迹和评估结果。最小 lineage 至少包含：repo URL、license、base commit、target commit、file hashes、采集时间、数据 split、构建镜像 digest、测试命令、模型/scaffold 版本和污染标签。

Listing 12: 建议的统一任务与轨迹记录（示意）

```
{
  "task_id": "repo@base_commit#issue_or_synthetic_id",
  "repo": {"url": "...", "base_commit": "...", "license": "..."},
  "problem": {"text": "...", "source": "issue|pr|synthetic"},
  "environment": {"image_digest": "sha256:...", "test_cmd": "..."},
  "oracle": {"patch_hash": "...", "fail_to_pass": [], "pass_to_pass": []},
  "trajectory": [{"observation": "...", "action": "...", "result": "..."}],
  "outcome": {"resolved": false, "failure_type": "localization"},
  "provenance": {"collected_at": "...", "split": "train", "contamination": []}
}
```

D.2 真实任务、合成任务与轨迹数据

真实 Issue/PR 的优势是意图、讨论和修改都来自开发过程，但存在关联错误、环境腐化、隐藏依赖、合并噪声与测试不足。合成任务更容易规模化并控制难度，但可能学习到生成器特征。SWE-Gym 提供 2,438 个带可执行环境的真实 Python 任务，并公开成功轨迹；其 rejection-sampling SFT 显示少量高质量多轮轨迹就能显著减少循环与空 patch[19]。SWE-smith 先为仓库建立共享环境，再用函数重写、程序化 mutation、组合 bug 和 PR mirror 合成任务，报告 128 个仓库上约 50K 实例，并用 5,016 条专家轨迹训练模型 [20]。R2E-Gym 使用测试生成和 commit back-translation 构造 8.7K 级环境，并研究执行式/非执行式 verifier 的互补性 [21]。SWE-rebench 则面向持续采集

和新鲜评估，公开超过 21K 个 Python 交互任务，并显式讨论污染 [22]。

由这些工作可归纳出四条工程规则：

1. **Environment first**：先证明 base commit 可安装且基线测试稳定，再生成或接入任务；
2. **双重验证**：gold patch 应实现 F2P，同时保持 P2P；重复运行以剔除 flaky test；
3. **成功与失败并收**：成功轨迹用于行为克隆，失败轨迹用于 preference、verifier、failure classifier 和 curriculum；
4. **按实例限额**：同一易题的大量成功采样会造成 easy-task bias，应按 task/repo cap 并报告去重后轨迹数。

D.3 轨迹质量与失败标签

轨迹应分解为可诊断阶段，而非只有最终 reward：问题理解、文件定位、符号定位、错误复现、方案生成、patch apply、局部测试、回归测试、终止判断。建议保留以下失败标签：env_setup、localization、reasoning、edit_syntax、test_generation、regression、loop、budget、unsafe_action、invalid_task。标签可由规则、执行信号和 LLM judge 联合产生，但 judge 只能做软标签，不能替代测试事实。

轨迹价值不应只按成功筛选。可定义：

$$v(\tau) = \alpha r_{\text{exec}} + \beta r_{\text{progress}} + \gamma d_{\text{novelty}} - \delta c_{\text{tokens}} - \eta r_{\text{risk}}, \tag{6}$$

其中 r_{progress} 可由定位正确、复现成功、减少失败测试数等 dense signals 构成；d_{novelty} 用 task/repo/action n-gram 或 embedding 多样性近似。式(6)是工程建议，并非公开论文的统一定义。

E Mid-training：从单文件模型迁移到仓库级编辑

E.1 Repo-level 组织的三个层次

仓库数据组织至少有三种粒度：

1. **同仓随机拼接**：保留 repo 边界和 file path，但不显式编码依赖；成本最低，是必要 baseline；
2. **结构感知拼接**：按 import、call、inherit、test-to-source、build dependency 排序或采样；
3. **任务条件组织**：围绕 target symbol/issue 构建局部子图，加入 hard negative 与 oracle context。

DeepSeek-Coder 与 StarCoder2 证明了 project/repository-level 序列可用于开放代码模型训练 [16, 15]；但“把同一 repo 文件塞入同一窗口”不自动证明模型学会了依赖关系。CrossCodeEval 明确要求跨文件上下文，并覆盖 Python、Java、TypeScript 和 C#[23]；RepoBench 则把 retrieval、completion 和 pipeline 拆分评估 [24]。因此，多层引用（DFS/BFS）、file-level、same-repo shuffled 和 dependency-aware 必须在等 token、等语言配比、等上下文长度下比较。

推荐的 8K/32K pack 由以下单元组成：目标文件局部上下文 + 直接依赖 + 测试/调用方 + 文档/Issue + 难负例。若窗口不足，优先保留声明、签名、类型、调用点和测试，而非完整实现。线上 runtime 的检索策略应与 mid-training 的上下文分布一致，否则训练时依赖 oracle context、推理时依赖低 recall 检索会产生明显分布偏移。

E.2 PR/Issue 作为 mid-training 信号

PR/Issue 数据连接“用户意图–仓库状态–修改–讨论–测试”，是从代码模型向编辑模型迁移的天然桥梁。建议形成多任务：

- Issue → changed files / symbols（定位）；
- Issue + local tree → inspection actions（检索策略）；
- before + intent → diff / str_replace（修改）；
- diff + source → tests（单测生成）；
- candidate patch + logs → critique / repair（自修复）；
- PR discussion → review decision / revised patch（代码审查）。

Clean-PR 的最新公开结果支持把经过严格筛选、patch 可应用且去污染的 PR 用作 mid-training 信号，但其具体收益依赖所用模型、过滤器和评估 scaffold[18]。工程上应先测 file recall@k、symbol recall@k、patch apply rate、edit exactness 和 held-out trajectory loss，再看端到端 resolve；这样能区分“学会定位”“学会编辑”和“scaffold 补救”。

F 长上下文扩展：目标是有效利用，不是配置值变大

F.1 位置扩展、数据适配与系统成本

RoPE 将第 m 个位置映射为按维度旋转的表示。对第 i 个二维通道，其相位可写为

$$\phi_i(m) = m\theta_i, \qquad \theta_i = b^{-2i/d}. \tag{7}$$

超出训练长度时，$\phi_i(m)$ 进入 OOD 区域。YaRN 通过频率分区、插值与 attention scaling 降低扩窗训练成本 [25]；LongRoPE 进一步搜索非均匀插值并采用渐进式扩展 [26]。这些方法解决的是位置表示与训练稳定性，不保证模型具备仓库级多跳推理。LongAlign 的结论也指出，长上下文模型需要相近长度的指令数据，并需处理 packing 与 loss weighting[27]。

标准全注意力的计算和激活随长度近似为

$$C_{\text{attn}} = \mathcal{O}(L^2 d), \qquad M_{\text{KV}} = \mathcal{O}(L\, n_{\text{layer}}\, d_{\text{kv}}), \tag{8}$$

因此从 32K 扩到 128K 不是单纯改 rope_theta：需要 sequence parallel/context parallel、FlashAttention、长度分桶、packing、checkpointing、KV cache 策略与推理预算共同设计。Code Agent 还应问：是否真的需要把整个 repo 放进窗口，还是用 32K–64K 高 recall 检索 + 迭代查看更划算。

F.2 建议的长上下文课程

表 4: 长上下文训练课程与门禁

阶段	长度分布	数据组织	门禁
L0 保能力	4K–8K 为主	file-level NTP/FIM + 通用/数学 replay	短上下文 benchmark 不显著回退
L1 位置适配	8K/16K/32K 混合	same-repo pack + 长文档	长度分桶 PPL、passkey、多 needle
L2 结构学习	16K–64K	dependency-aware、多层引用、target-aware	CrossCodeEval/RepoBench 随有效 context 提升
L3 指令对齐	32K–128K 少量	长 Issue、目录、轨迹、multi-file diff	长程定位、计划保持、工具协议不退化
L4 推理优化	真实分布	检索压缩、KV/reuse、迭代浏览	同成功率下降低 latency/token cost

训练 mixture 不能只用最大长度。建议每个 batch bucket 明确 token 占比，保留 30%–60% 短序列 replay 作为初始探索范围，并以实验调节。RULER 表明 vanilla needle 准确率可能掩盖 multi-hop 和 aggregation 退化 [11]，所以长代码评估还必须加入跨文件引用链深度、干扰文件数量、target 位置和 repo size 分桶。

G 后训练：从“会回答”到“会行动并验证”

G.1 SFT：先学稳定协议与基本策略

SFT 数据至少覆盖三类：原子工具技能、结构化工作流和自由多轮轨迹。原子技能包括 search/view/edit/test/git diff；结构化工作流包括定位–修改–验证；自由轨迹覆盖根据执行反馈动态修正。CodeAct 用可执行 Python 统一动作空间，并公开 7K 多轮交互数据，说明“动作表示”本身会影响 Agent 学习 [28]。SWE-agent 强调 ACI 对导航、编辑和测试行为的影响 [6]。

训练 mask 应区分 system/tool schema、observation、reasoning 和 action。若产品不输出显式思维链，可将可审计的短计划、状态摘要和动作理由作为训练目标，而不依赖长篇自由推理。SFT 关键指标包括 tool-call parse rate、action validity、empty patch、loop rate、平均 turns、无效读取比例和 test-before-finish 比率。

G.2 偏好优化与 verifier

偏好对可由同题多轨迹构造：成功优于失败；无回归优于只过新增测试；更小且解释充分的 patch 优于大范围改写；相同成功下低成本优于高成本。DPO 可在不在线采样的情况下学习相对偏好，但若 preference 主要由 judge 产生，模型会学习 judge 风格和长度偏差。应混合执行事实、人工 rubric 与模型评分，并单独评估 reward hacking。

Outcome verifier 对轨迹 τ 和 patch P 估计

$$s_\phi(\tau, P) = P_\phi(\text{YES} \mid \text{task}, \tau, P), \tag{9}$$

用于 Best@k 选择或过程 pruning。SWE-Gym 使用成功/失败轨迹训练 outcome-supervised verifier[19]；R2E-Gym 发现 execution-based 与 execution-free verifier 各有盲区，组合后更有效 [21]。因此 production verifier 应由回归测试、定向测试生成、静态分析、diff risk 与 learned score 共同组成，而不是单一 reward model。

G.3 为什么 Code Agent RL 不是普通的单轮 RLVR

RL 是 Code Agent 从“模仿成功轨迹”走向“依据环境反馈主动探索”的关键模块，但软件工程任务同时具有**状态可变、轨迹极长、奖励稀疏、环境昂贵且会失败**四个特征。数学/竞赛代码 RLVR 通常只需对一个完整输出判分；Code Agent 则要在几十轮工具交互和数万至十余万 token 后，才由隐藏测试给出终局奖励。Long-Context Multi-Turn SWE RL 的公开实验把这种差异形式化为 stateful MDP，并指出把一个 terminal advantage 广播给整条轨迹会产生严重信用分配噪声 [29]。

因此，训练前应满足三个先决条件：其一，SFT/RFT 初始策略在训练题上已有非零 pass@k，否则同组 rollout 全为零，group-relative advantage 不可用；其二，gold patch、空 patch 和重复运行能验证 evaluator 的确定性；其三，容器启动、依赖安装、测试 flaky 等基础设施失败能够与 policy failure 分离。RL 提升的直接对象不是“代码知识总量”，而是搜索顺序、试验选择、失败恢复、终止与提交这些闭环策略。

G.4 四条已被实践验证的技术路线

G.4.1 路线 A：基于软件演化数据的 proxy-reward RL

SWE-RL 从 GitHub Issue/PR 构造 issue、pre-change context 与 oracle patch，使用预测 patch 和 oracle patch 的序列相似度作为连续 reward，并以 GRPO 优化 [30]。这种方法不需要启动完整仓库环境，吞吐高、易扩展，连续相似度也比“是否完全匹配”的离散 reward 更密集。其局限同样明确：patch 文本相似不等价于语义正确，会惩罚与开发者实现不同但有效的 patch；且其主训练任务给定相关代码上下文，弱化了真实 Agent 的定位与多轮执行难度。它更适合做大规模 reasoning/editing warm-up，而不应作为最终 agentic reward。

G.4.2 路线 B：真实执行环境中的长轨迹 RL

执行型 RL 直接以 F2P/P2P 测试判定 patch。Golubev 等人在 131K context 的多轮环境中采用 RFT 冷启动和修改后的 DAPO：每题采样 10 条完整轨迹、组内归一化 terminal reward、丢弃零 advantage 组，并加入轨迹长度惩罚；在相同 scaffold 下从 20% 的 RFT baseline 提升到 39% SWE-bench Verified[29]。论文也警告：事后截断超窗轨迹会造成 rollout policy 与训练 log-prob 不一致，破坏 importance sampling 假设。

SWE-Master 给出了更工程化的稳定策略 [31]：先对约 60K 条长轨迹 SFT，再在 Docker 环境中做 GRPO；以 leave-one-out advantage、固定长度归一化、clip-higher 和去 KL 维持探索；对预算耗尽轨迹强制提交当前 patch，并按 stop reason 折扣 reward；容器/服务失败直接 mask，不进入 policy update；每轮 observation 显式返回剩余预算。其重要启发不是某个单独超参，而是 **termination semantics 和 infrastructure semantics 必须进入 RL 数据模型**。

G.4.3 路线 C：用 guidance 或子能力分解降低稀疏性

Agent-RLVR 让 Agent 先自行尝试，对失败轨迹再提供 plan、错误纠正或环境交互提示，重新 rollout 后用可验证 reward 更新；guidance 只在训练采样时出现，推理时移除 [32]。其 Qwen2.5-72B-Instruct 在 SWE-bench Verified 上由 9.4% 提升到 22.4%，再用同批轨迹训练 reward model 做候选排序达到 27.8%。这说明 teacher 的价值可以是让稀疏奖励区域出现成功样本，而不一定把完整答案蒸馏给学生；但需要检查模型是否依赖 guidance 风格，以及 teacher 是否泄露 gold 信息。

另一种做法是先训练可密集判分的原子策略。CodeScout 把 code localization 定义为 file/module/function 三粒度集合预测，使用三者 F1 之和作为 reward，Agent 仅用 Unix terminal 搜索并通过结构化 finish tool 提交 [33]。这一设计把 reward 从“最终 patch 是否完全正确”变为“相关位置找得多准”，同时避免脆弱字符串解析。子能力 RL 应与端到端 RL 串联验证：定位 F1 上升只有在固定 repair model、固定 budget 下提升 resolve rate，才算形成有效迁移。

G.4.4 路线 D：自博弈生成难度递增的可执行课程

Self-play SWE-RL 让同一 policy 交替扮演 bug injector 和 solver：注入端从真实仓库生成 bug patch 与 test-weakening patch，通过一致性检查过滤不可执行样本；solver 在隐藏测试约束下修复；注入 reward 鼓励“有效、非平凡、但仍可解”的 bug，而不是越难越好 [34]。该方法在不依赖人类 Issue/PR 标注的设置中报告 SWE-bench Verified 和 SWE-Bench Pro 分别提升 10.4 与 7.8 个点。其关键风险是自博弈分布退化：注入器可能学习人工化、易判分却不符合真实缺陷的模式，因此必须监控 bug type、修改范围、语言/repo 分布及对真实 Issue 的迁移。

表 5: Code Agent RL 路线的信号、优势与主要偏差

路线	主要 reward	适合解决的问题	主要偏差/门禁
Proxy patch RL	oracle patch 相似度、格式	低成本 reasoning/editing warm-up	等价 patch 误罚；必须用执行评测复核
Execution RL	F2P/P2P、回归与终止折扣	端到端搜索–编辑–验证策略	环境吞吐、flaky、稀疏奖励与长程信用
Guided / skill RL	guided success、定位 F1、过程里程碑	冷启动、定位/测试等瓶颈能力	guidance 泄露、proxy 与 resolve rate 脱钩
Self-play RL	bug 有效性、目标难度、修复成功	扩展任务分布和自动课程	合成缺陷模式坍缩；需真实任务迁移门禁

G.5 奖励、优化与信用分配的可落地配方

执行 reward 不应只保留一个 success bit。建议保留可审计的向量，再在训练侧组合：

$$r(\tau) = \alpha r_{\text{F2P}} + \beta r_{\text{P2P}} + \gamma r_{\text{progress}} + \eta r_{\text{submit}} - \lambda_1 c_{\text{turn}} - \lambda_2 r_{\text{unsafe}} - \lambda_3 r_{\text{test tamper}} - \lambda_4 r_{\text{scope}}. \tag{10}$$

r_{progress} 只能来自较难投机的证据，如编译错误减少、目标测试通过子集增加、复现测试从 pass 变 fail、静态类型错误收敛；不能直接奖励“调用了测试”“读取了文件”等表面动作。r_{submit} 区分主动提交和预算耗尽后的强制提交；r_{scope} 约束无关文件和 diff 扩张。删除/放宽测试、硬编码样例、修改 grader、绕过依赖、写环境外部状态都应由独立 rule-based detector 触发 hard zero 或样本剔除。

优化上可采用 GRPO/GSPO/DAPO 类无 critic 方法作为起点，但要记录 group success diversity：若同组全成功或全失败，更新信号为零；若高难题长期全零，应提高 rollout 数、加入 guidance/课程或回退到 rejection fine-tuning，而不是盲目拉长训练。对超长轨迹，单一 trajectory advantage 广播到所有 token 只是基线。进一步方案包括：（1）从共享 prefix 分叉 rollout，比较后续动作的反事实价值；（2）用可验证 milestone 产生 step/segment reward；（3）训练 critic/value head 做局部 advantage；（4）只对模型 action token 求 loss，mask tool observation 和基础设施异常。过程 reward 越密，越要检查是否诱导“刷进度”而非完成任务。

G.6 RL 系统、数据血缘与实验归因

训练系统至少包含 environment pool、rollout workers、trajectory/event store、reward/evaluator、trainer 和 checkpoint broadcaster。每条样本必须绑定 repo commit、container digest、test patch、harness/scaffold commit、model revision、sampling seed、stop reason、reward components 与 artifact URI。异步 rollout 能提高 GPU/CPU 利用率，但旧 checkpoint 产生的 trajectory 会增加 off-policy staleness；应记录 policy version 并设置最大滞后，而不是把所有轨迹混为同一分布。

最小因果实验矩阵应固定 base/SFT checkpoint、task split、harness、context、turn/token budget 和 rollout 总数，比较：SFT/RFT、offline preference、proxy-reward RL、execution RL、execution RL + guidance、execution RL + process signal，以及不更新参数的 best-of-k + verifier。主指标除 resolve rate 外，还应报告 pass@k、group nonzero rate、reward/entropy、active-submit rate、forced-submit success、environment mask rate、平均 turns/tokens、diff size、回归/越权率和 unseen-repo 迁移。只有在**相同采样预算**下胜过 SFT 与 test-time scaling，且在 held-out repo 上保持收益，才能把提升归因于 policy learning。

H Agent Runtime：让模型在受控接口中工作

H.1 最小而充分的工具面

基础工具建议限制为：目录/符号搜索、文件分段读取、结构化编辑、终端执行、测试、diff/status 与最终提交。工具应返回稳定、可解析且长度受控的 observation；编辑工具必须验证 old text 唯一匹配或基于 patch apply；终端需限制网络、文件系统和进程权限。OpenHands 将终端、编辑器与浏览器等置于沙箱平台 [7]，但在企业代码场景还需增加 secret redaction、依赖源 allowlist、审计日志和人工审批门。

运行时不应默认“全 repo 注入”。更稳健的上下文管理包括：repo map、symbol graph、BM25/embedding 混合检索、最近 observation 压缩、persistent scratchpad 与 diff-aware cache。Agent 每轮应得到当前目标、已知事实、工作区状态和剩余预算，而不是不断拼接原始历史。

H.2 Harness：连接模型策略与可执行世界的控制平面

Harness 不等于 scaffold，也不等于 evaluation harness。 Scaffold 定义模型看到的 prompt、工具与行动风格；agent harness 负责把一次任务可靠地运行成完整 episode；environment 提供仓库与执行隔离；evaluation harness 则在 episode 结束后应用 patch、运行标准测试并判分。一个结果应明确写成

$$Y = f(M, S, H, E, V, B, \xi), \tag{11}$$

其中 M 为模型 checkpoint，S 为 scaffold/prompt，H 为 agent harness，E 为环境镜像，V 为 evaluator/verifier，B 为预算，ξ 为采样随机性。只报告“模型名 + SWE-bench 分数”无法复现式(11)中的其余变量。

Agent harness 应实现一个显式 episode 状态机：INIT → OBSERVE → MODEL_STEP → PARSE/POLICY → EXECUTE → UPDATE，在成功、失败、超时、预算耗尽、越权或人工中断时进入终态。每个转移都要幂等或可判定是否已执行，避免模型/API 重试造成同一写操作重复发生。

表 6: Code Agent harness 的核心模块与验收点

模块	职责	必测故障与指标
Episode Orchestrator	状态机、终止条件、重试、checkpoint/resume	重复执行、死循环、恢复一致性、terminal reason 分布
Model Adapter	统一 chat/response/tool-call、流式输出与 token accounting	schema 差异、截断、rate limit、实际输入/输出 token
Context Manager	history、repo map、检索、压缩、cache 与 context budget	信息丢失、超窗、cache stale、有效 context 命中率
Action Parser/Policy	解析动作、参数校验、allow/deny/approval、去重	parse rate、invalid action、越权阻断、审批次数
Environment Adapter	local/container/remote sandbox、workspace 与进程生命周期	base commit 漂移、僵尸进程、网络/文件越界、资源泄漏
Tool Runtime	search/view/edit/shell/test/git 的超时、输出截断和错误映射	command timeout、partial output、patch 冲突、工具错误率
Budget Controller	turns、tokens、wall time、cost、CPU/GPU/IO 限额	超预算动作、预算估计偏差、成功成本分布
Event Store	append-only event、artifact、diff、日志、trace 与 replay	丢事件、敏感信息、trace 可重放率、版本字段完整率
Evaluator Bridge	生成标准 prediction、提交 evaluation harness、收集细粒度结果	patch serialization、F2P/P2P、gold sanity、评测超时

Harness 的设计复杂度应与研究问题匹配。SWE-agent 适合研究工具集合、history processor 和 ACI；mini-SWE-agent 则用线性消息历史、独立 shell action 和极小控制流，把研究焦点放回模型本身 [35]。两者没有绝对优劣：复杂 harness 便于加入 guardrail、检索与恢复，最小 harness 更适合作为模型能力 baseline，并降低 scaffold overfitting。

评估侧必须与 agent harness 解耦。SWE-bench 官方 evaluation harness 采用分层 Docker image，执行 setup、patch application、test、grading 和 reporting，并提供 cache、timeout 与日志控制 [36]。内部评估也应先用 gold patch 做 harness sanity check，再测空 patch/错误 patch 的负控；否则环境或 grader 故障会被误记为模型失败。每次 run 至少冻结以下 manifest：

Listing 13: Harness run manifest 的最小复现信息

```
run_id, task_set_version, model_id, model_api_revision,
scaffold_commit, harness_commit, prompt_hash, tool_schema_hash,
environment_image_digest, evaluator_commit, retriever_index_digest,
context_limit, max_turns, max_tokens, timeout_s, sampling_seed,
network_policy, permission_policy, cache_policy
```

对训练数据生产，harness 还要支持 deterministic replay 和 counterfactual replay：前者在冻结 observation 时重放 model/actions，检查 parser 与状态机一致性；后者替换模型或某一步 action，比较后续分支。若 observation 含时间、网络或并发等非确定状态，应保存原始事件和外部 artifact，而不是假设命令可再次产生相同输出。

H.2.1 Claude Code：从公开接口重建 Harness 架构剖面

Claude Code 的完整内部实现并未公开，因此不能把产品行为反推成确定的私有模块图。可由官方文档和工程文章确认的，是一组公开控制面：模型在环境反馈驱动的 loop 中选择工具；Claude Agent SDK 暴露与 Claude Code 同源的工具、context management 和 permission primitives；项目指令、memory、compaction、subagent、hooks、

MCP、permissions 与 sandbox 分别承担不同控制职责 [37, 38, 39]。据此可得到表7，它是公开能力的工程映射，不是对闭源实现的逆向结论。

表 7: Claude Code 的公开 Harness 控制面及可迁移设计原则

控制面	Claude Code 公开机制	Harness 职责	对自建 Code Agent 的启发
上下文/记忆	CLAUDE.md 分层指令、auto memory、compaction	注入项目规范，压缩历史，跨会话保留可复用事实	指令、工作记忆、长期记忆分库存储并记录 provenance[40]
工具/扩展	Read/Edit/Bash、MCP、Skills	统一 tool schema、observation 截断与外部连接	工具能力与权限分离；外部内容视为不可信输入
确定性控制	lifecycle hooks、settings、finish/stop 条件	格式化、审计、阻断保护文件、压缩后重注入	硬约束放 hook/policy，不依赖 prompt 自觉 [41]
任务隔离	subagent 独立 context、tool/model/permission 配置	隔离高噪声检索/测试，返回结构化摘要	delegation 必须记录输入摘要、agent 配置和 handoff artifact[42]
安全边界	permission 先判工具；Bash 再受 OS sandbox 的文件/网络约束	allow/ask/deny、最小权限、egress 与 workspace 隔离	policy guard 与 OS enforcement 叠加，不能互相替代 [43]
可恢复执行	session resume、checkpoint、structured handoff	跨 context/session 恢复状态与未完成任务	以文件/事件为真相源，恢复后先验证环境而非相信摘要

长任务 Harness 的公开演进也很有启发。Anthropic 早期方案用 initializer agent 建立环境、任务清单和进度文件，再让 coding agent 每个 session 做增量工作并留下 handoff artifact，解决 context reset 后的失忆 [44]；后续应用构建方案扩展为 planner–generator–evaluator，生成器与评估器在每个 sprint 前协商可测试 contract，评估器用浏览器/执行反馈验收 [45]。这两者是建立在 *Claude Agent SDK* 上的示例 *Harness*，不应写成 Claude Code 默认内部拓扑。它们共同说明：长程自治的关键状态应落在 repo、任务表、测试契约和 event log 中，而不是只留在聊天上下文。

安全面同样体现双层控制。公开的 Claude Code sandbox 对 Bash 子进程使用 macOS Seatbelt 或 Linux bubblewrap 做 OS 级文件系统约束，并通过 sandbox 外代理实施域名 egress 控制；permission 则在工具运行前决定 allow/ask/deny，两者覆盖面不同 [43]。Auto mode 的公开架构又增加输入侧 prompt-injection probe 和输出侧 tool-call classifier，subagent 递归经过相同管线 [46]。对企业 Code Agent，等价实现至少应区分：**模型倾向约束、Harness policy、OS/容器强隔离、外部连接器权限**，并用攻击回放分别测每层是否真正生效。

Harness 设计边界: Harness 能显著抬高特定模型的任务成功率，但它编码了“模型当前做不到什么”的假设。模型升级后，应逐项移除 planner、强制阶段、额外 verifier 或 context reset 做消融；若分数不降，就删除该复杂度。反过来，任何声称模型能力提升的实验，都应先在 minimal harness 与 production harness 两个剖面复测，防止把 scaffold/harness overfitting 误记为 checkpoint 收益。

H.3 自治程度与工作流选择

不同应用应选择不同自治等级：

- **L0 建议**：补全、解释、review comment，不能直接写仓库；
- **L1 受控编辑**：生成 patch，用户确认后执行；
- **L2 沙箱任务**：自主搜索、编辑和测试，但不能访问生产资源；
- **L3 工作流 Agent**：可建分支、提交 PR、响应 CI，关键动作审批；
- **L4 长程自治**：跨任务规划与持续运行，仅适合高可验证、强隔离场景。

Agentless 的结果说明，对于定位、修复、验证可明确分解的任务，固定 pipeline 可能更便宜且可解释 [12]。因此自治不是默认目标，而是当动态环境与分支决策确实带来收益时才逐级开放。

I 评估体系：必须把模型、检索、scaffold 与预算解耦

I.1 四级评估矩阵

表 8: Code Agent 分层评估矩阵

层级	代表任务/基准	核心指标	解释边界
函数级	HumanEval、MBPP、LiveCodeBench、BigCodeBench、DS-1000	pass@1/pass@k、执行正确率、库调用覆盖	测语法、算法、指令和 API 使用；不能代表跨文件定位 [47, 48, 49, 50]
跨文件	CrossCodeEval、RepoBench、FIM/repair suites	EM、Edit Similarity、CodeBLEU、retrieval recall@k	应报告有/无 oracle context 与语言分项；不能直接代表多轮 Agent[23, 24]
仓库任务	SWE-bench Verified/Multilingual、SWE-bench-Live、SWE-rebench	resolve rate、patch apply、F2P/P2P、成本	高度依赖环境、scaffold、预算与污染；必须固定版本 [5, 22, 51]
产品闭环	私有 Issue、代码审查、迁移、测试生成、CI 修复	接受率、time-to-merge、revert、人工接管、风险、成本	最接近业务价值，但需对任务难度、用户与仓库分布做归一化

pass@k 的无放回估计为 [47]

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \tag{12}$$

其中 n 是采样数、c 是正确样本数。对 Agent，应同时报告 pass@1、pass@k 和 verifier-selected Best@k；三者对应单次策略、多样性上限和选择器能力，不能混写成同一分数。

I.2 公平比较与污染控制

端到端比较至少固定：base model checkpoint、context limit、prompt/scaffold、工具集合、检索器、最大 turns、token/美元预算、temperature、容器镜像、并发与重试。训练实验还必须固定总训练 token、语言配比、采样次数、序列长度分布与 optimizer steps。若 repo-level 方案训练了更多 epoch 或更多实际 token，loss/benchmark 上涨不能归因于组织策略。

污染治理建议四层并行：repo exclusion；文件/patch exact hash；代码 10–15 gram 或 MinHash 相似度；Issue/描述 embedding/Jaccard。时间切分以 Issue/PR 创建时间和模型 data cutoff 双重标记。LiveCodeBench 通过持续收集新竞赛题减轻静态代码基准污染 [48]；SWE-rebench 与 SWE-bench-Live 将“持续采集新鲜可执行任务”扩展到软件工程 Agent[22, 51]。新鲜评估并不自动无污染，还需检查 repo 历史、派生 patch 和基础模型暴露。

I.3 推荐 dashboard

至少同时展示：

- **能力**：resolve、F2P、P2P、file/symbol recall@k、tool-call validity；
- **效率**：tokens/task、turns/task、wall time、GPU/CPU/container hours、成功任务成本；
- **行为**：loop、empty patch、test execution、rollback、无效动作与 context utilization；
- **质量**：diff size、回归、静态告警、review 接受率、revert rate；
- **数据**：unique/actual token、dedup ratio、repo/language/task diversity、环境构建率；
- **风险**：secret exposure、越权、依赖供应链、测试篡改与 license violation。

J 应用场景：按可验证性与风险选择落点

优先落地应选择 reward 清晰、环境隔离、失败可回滚且人类审阅成本可控的场景。Issue-to-PR 很适合研究，但生产首站往往是定位、测试生成或受控 patch，因为它们更容易将模型收益与系统风险解耦。对安全修复、依赖升级和 CI，应把网络访问与发布权限放在独立执行器，不让语言模型直接持有长期凭据。

表 9: 应用场景、训练信号与上线门槛

场景	关键能力/数据	主评估	建议自治级别
IDE 补全与编辑	FIM、局部上下文、用户接受/撤销	接受率、edit survival、延迟	L0–L1
Repo 问答与定位	repo map、符号图、Issue-to-files	recall@k、回答可引用性	L0–L1
Bug 修复/Issue-to-PR	PR/Issue、执行轨迹、回归测试	resolve、P2P、人工接管	L2–L3
测试生成	source-to-test、mutation、覆盖反馈	mutation score、缺陷发现率	L1–L2
代码审查与安全修复	PR discussion、静态扫描、CVE	precision/recall、误报、风险等级	L0–L2
重构/迁移	多文件 diff、build graph、API 版本	build/test、语义等价、diff 风险	L2–L3
数据科学与分析自动化	Notebook、DS-1000、库文档、执行	execution、结果正确性、可复现	L1–L2
CI/DevOps 修复	build log、workflow、依赖图	pipeline recovery、供应链风险	L2，关键动作审批

K 实验路线图：用等 token 与分层指标建立因果证据

K.1 阶段 A：代码基座与数据 pipeline

目标是证明数据质量与配比收益，而不是追求最大模型。建议先用 1B–3B proxy，固定 architecture、tokenizer、训练 token 和 batch token，比较：

1. file exact + fuzzy dedup 的阈值与误杀率；
2. code-only vs code+text vs code+text+math；
3. rule filter、model filter、value filter 的独立和组合增益；
4. code-related Web、Issue/PR/Commit 的增量价值；
5. FIM rate 与多语言采样温度。

门禁：每个候选同时给出 held-out loss trajectory、HumanEval/MBPP/LiveCodeBench、BigCodeBench、语言分项、unique token 和误杀抽检。训练效率应按达到同等 benchmark 的 token 数，而不只看最终分数。

K.2 阶段 B：Repo-level 与长上下文

核心 $2\times2\times2$ 消融为：file vs repo；same-repo shuffle vs dependency-aware；1-hop vs multi-hop。固定 8K 与 32K 实际训练 token，再增加 64K 小比例课程。报告：

- file→func 组织成功率、func→file 还原一致率、遗漏 symbol/file 数；
- CrossCodeEval 四语言、RepoBench retrieval/completion/pipeline；
- target dependency depth、repo size、context length 和 target position 分桶；
- algorithm benchmark 安全线与短上下文回退；
- oracle context、训练同构 retriever 和真实 retriever 三种 setting。

如果 dependency-aware 只在 oracle context 下有效，应优先修检索；如果 same-repo shuffle 与 dependency-aware 接近，说明收益可能来自位置适配或同域长序列，而非图结构。

K.3 阶段 C：轨迹学习与 Agent

先固定一个最小 scaffold，在相同 32K context、turn budget 和 temperature 下比较：base、atomic-tool SFT、successful-trajectory SFT、success+failure preference、verifier Best@k、online RL。训练数据以 task/repo 去重后的轨迹数和 actual tokens 双报。建议关键消融包括：

- 有/无 reasoning summary；有/无 execution observation；
- 真实 Issue vs PR mirror vs procedural/synthetic bugs；
- 单成功轨迹/题 vs 多轨迹/题；随机扩量 vs 新 repo/新任务扩量；
- outcome verifier vs process verifier vs execution verifier；
- 32B 单次采样 vs 较小模型 Best@k 的等成本比较。

表 10: 建议的阶段性 go/no-go 门槛（需按现有 baseline 校准）

阶段	Go 条件	No-go 信号	下一动作
数据 pipeline	等 token 下多项代码评估稳定增益，误杀可解释	仅 loss 下降或只涨单一 benchmark	做污染/重复审计与语言分桶
Repo mid-train	跨文件指标提升且算法安全线不退	只在更长输入或 oracle context 下提升	加 same-repo shuffle 与 retriever 控制
长上下文	多 hop/aggregation 随长度保持，短任务不退	仅 passkey 成功，真实 repo 无增益	调整长数据与位置课程
轨迹 SFT	resolve、loop、empty patch 同时改善	成功率涨但成本/回归恶化	加失败轨迹与成本/风险偏好
RL/verifier	等 rollout 成本优于 SFT/best-of-k	reward 上升但私有集不涨	查 reward hacking 与环境噪声
上线	私有任务接受率、回滚与风险达标	benchmark 高但人工接管/成本过高	降自治级别、收窄场景

L 主要风险与开放问题

L.1 数据与法律边界

公开可访问不等于可随意训练或再分发。应记录 repo/file 级许可证、删除请求、PII/secret 检测、生成代码归因与训练数据 provenance。The Stack v2 的 Software Heritage 标识与许可证流程提供了可审计范式 [15]，但不同组织仍需结合使用地区和产品形态做法律评估。

L.2 环境与测试不等于真实正确性

测试可能不完整、脆弱或可被投机；旧仓库环境也可能因依赖消失而不可重现。任务构建率、基线稳定率、gold patch 验证率必须作为数据质量指标，而不是隐藏在 dataset size 后。SWE-smith、SWE-rebench 等工作都把环境构建视为主要成本与误差源 [20, 22]。

L.3 基准饱和与系统共适应

同一批仓库长期出现在训练、prompt 设计和 leaderboard 中，会导致模型、检索器和 scaffold 对基准共适应。应维护按月/季度滚动的 shadow set、私有任务和时间外推集，并冻结一套可复现 baseline。任何 SOTA 数字都应附带模型版本、scaffold commit、预算和日期。

L.4 能力外推与安全

从“能修 Python Issue”不能外推到多语言、大型 monorepo、分布式系统或生产发布。长程 Agent 还可能执行破坏性命令、泄露 secret 或引入供应链依赖。安全策略必须由沙箱、权限系统和 policy engine 强制执行，不能只依靠 prompt。

M 总结与展望

构建 Code Agent 的正确主线，是把代码模型训练与可执行软件工程环境统一起来。预训练解决“懂代码与修改模式”，mid-training 解决“理解仓库结构并利用长上下文”，后训练解决“按协议行动、从反馈修正”，runtime 解决“安全、低成本、可审计地执行”，持续评估则保证这些收益在新鲜任务上仍然成立。

短期最有确定性的投入不是盲目扩大模型或窗口，而是三项基础设施：第一，带许可证、commit、测试与 split 血缘的统一数据层；第二，可批量构建、缓存和复验的环境层；第三，固定 scaffold/预算、能拆分定位–编辑–验证的评估层。有了这三层，预训练数据策略、repo-level 组织、轨迹 SFT、verifier 与 RL 才能通过等 token、等成本实验获得可信归因。

中期的研究重点应从“更多成功轨迹”转向“覆盖更多 repo、语言、失败模式和依赖深度”，从“标称 128K”转向“在复杂干扰和多跳引用下的有效上下文”，从“单一 resolve rate”转向能力、成本、回归、安全和人工接管的联合 Pareto 前沿。长期看，Code Agent 的竞争力将更多来自数据与环境闭环，而非单次模型发布：系统能否把真实开发反馈快速转成干净、可执行、去污染的训练与评估实例，将决定迭代速度和可持续上限。

A 附录 A：数据与环境验收清单

表 11: 生产数据 pipeline 的最小验收字段

对象	必备字段	核心指标/检查
Repository	URL、license、default branch、snapshot/commit、language、stars/time	抓取成功率、许可证覆盖、fork/镜像关系、时间分布
File/Symbol	path、hash、language、generated/vendor/test/config、AST/symbol refs	parse 率、file/function 数、exact/fuzzy dedup、误杀抽检
Issue/PR/Commit	ID、时间、关联边、作者/bot、before/after、discussion、CI	关联准确率、patch apply、changed files、bot/noise ratio
Environment	image digest、OS、依赖、安装与测试命令、network policy	build 成功率、重复运行稳定率、大小、构建时延
Task	base/target commit、problem、gold/test patch、F2P/P2P、difficulty	有效任务 yield、flaky、测试区分度、污染标签
Trajectory	model/scaffold、prompt、actions、observations、diff、cost、outcome	成功率、loop、turns/tokens、failure taxonomy、多样性
Split	repo/time/hash/similarity rules、cutoff、evaluation exposure	repo overlap、code n-gram、Issue similarity、版本可追踪

B 附录 B：端到端实验记录模板

每个实验建议固定记录以下内容，避免只留下结果表：

1. **Hypothesis**：预期改变哪一个中间能力，为什么；
2. **Control**：相同 architecture、init、token、language mix、context bucket、optimizer；
3. **Data**：unique/pasted/actual trained tokens，repo/file/function/sample 数，版本与过滤规则；
4. **Training**：global batch tokens、steps、LR、FIM rate、packing efficiency、硬件与 wall time；
5. **Offline metrics**：loss trajectory、函数/跨文件/仓库/语言分项；
6. **Agent setting**：scaffold commit、tools、retriever、context、turn/token/cost budget；
7. **Statistics**：至少多 seed 或任务 bootstrap CI，列出失败类型变化；
8. **Decision**：ship/iterate/stop，以及下一条可证伪假设。

C 附录 C：公开方案的证据边界

表 12: 本文使用公开工作的方式与不可外推部分

公开工作	可支持的结论	不应直接外推
OpenCoder / StarCoder2	代码清洗、去重、许可证、code-related 数据的可复现做法	特定过滤阈值或配比适用于所有内部数据
DeepSeek-Coder / Qwen2.5-Coder	project/repo FIM 与阶段化长上下文训练可行	标称窗口等于真实仓库级 Agent 能力
YaRN / LongRoPE / LongAlign	位置扩展与长指令适配的机制与成本	无需任务型长数据或检索即可处理任意 repo
SWE-Gym / SWE-smith / R2E-Gym	可执行任务与轨迹能训练 Agent/verifier，且可规模化	公开 pass@1 可在不同 scaffold/预算间直接比较
SWE-bench / Live variants	真实 Issue 修复和持续评估的可执行范式	单一 Python/固定仓库分数代表完整 SDLC
Clean-PR / OctoPack	Commit/PR 可提供修改与意图监督	原始开发历史无需严格过滤即可训练

D 附录 D：关键阶段的数据样例与轨迹协议

本附录给出可直接落成 JSONL/Event Store 的**脱敏示意 schema**。所有 repo 名、commit、Issue 文本、路径、hash、测试数和 reward 数值均为本文虚构，**没有从公开数据集中逐条摘录**。样例只在对象关系和训练语义上参考公开论文、数据集与工具格式，并加入本文建议的 lineage、artifact、版本和安全字段。生产环境中，大段源码、命令输出和 patch 应保存为 content-addressed artifact，事件只保留摘要、hash 与 URI；下列示例为便于阅读进行了截断。

D.1 数据来源与改编边界

表13列出每组样例的公开依据与本文扩展。这里的“参考”表示概念和字段语义的改编，不表示公开来源采用了完全相同的 JSON key 或事件层级。

表 13: 附录 D 数据样例的来源依据与本文扩展

样例	公开依据	本文自行设计或扩展的部分
Repo-level/FIM	The Stack v2/StarCoder2 的仓库来源与可追溯标识，以及 FIM 的数据变换与 PSM/SPM 训练方式 [15, 52]	dependency_bfs、symbol relation、repo cluster、统一 split 对象
PR/Issue 原子任务	SWE-bench 的 Issue–base commit–patch–test 任务结构，CommitPack/OctoPack 与 Clean-PR 的开发历史监督 [5, 17, 18]	lineage_id、多视图 task type、细粒度 loss mask 与 quality gate
成功/恢复轨迹	SWE-agent 的 thought–action–observation .traj、SWE-Gym 的公开 Agent 轨迹，以及 ATIF 的完整交互记录思想 [53, 19, 54]	append-only event taxonomy、单调 seq、artifact/tree hash、显式恢复计数
偏好对	SWE-Gym 与 R2E-Gym 使用成败轨迹训练 verifier、比较执行结果的做法 [19, 21]	同 harness/预算配对约束，以及 diff size、回归覆盖、成本组成的偏好理由
在线 RL rollout	多轮 SWE Agent RL 中的组内 rollout、可执行测试 reward、终止语义、环境故障 mask 与 guidance 机制 [29, 31, 32]	policy staleness、reward vector、infra 状态与 token-level loss mask 的统一落盘协议
危险负轨迹	SWE-bench 的 test patch/评测 Harness 与 SWE-rebench 的污染和可复验约束 [5, 36, 22]	test_tamper/scope_violation 规则标签及 policy/verifier 训练用途

D.2 预训练与 mid-training 样例

预训练样本的关键不是只有一段 text，而是保留 repo/commit、文件关系、去重和 split 血缘。这样才能对 file-level random pack、same-repo pack、dependency-aware pack 与 FIM 做等 token 消融。

Listing 14: Repo-level 预训练/FIM 样例

```
{
  "sample_id": "pt_repo_01J8K7",
  "stage": "pretrain",
  "objective": "fim",
  "repo": {"id": "acme/cachelib", "commit": "8f31c2a",
           "license": "Apache-2.0", "language": "python"},
  "segments": [
    {"path": "src/cache.py", "symbol": "Cache.get",
     "relation": "target", "artifact": "sha256:aa91..."},
    {"path": "src/store.py", "symbol": "Store.read",
     "relation": "callee", "artifact": "sha256:bc27..."}
  ],
  "packing": {"strategy": "dependency_bfs", "tokens": 7312},
  "dedup": {"file_hash": "f19...", "repo_cluster": "rc_083"},
  "split": {"time_cutoff": "2025-01-01", "eval_overlap": false}
}
```

mid-training 样本应把“输入上下文”“原子能力目标”和“真实变更血缘”分开。一个 PR 可以派生 localization、structured edit、test generation 和端到端 repair 多种视图，但必须共享 lineage_id，防止同一变更跨 train/eval 泄漏。

Listing 15: PR/Issue 派生的 mid-training 原子任务

```
{
  "task_id": "mt_edit_0042", "lineage_id": "pr_1847",
  "stage": "midtrain", "task_type": "localize_then_edit",
  "input": {
    "issue": "Cache.get ignores ttl=0 and returns stale values.",
    "base_commit": "8f31c2a",
    "repo_map": ["src/cache.py:Cache", "tests/test_cache.py"]
  },
  "targets": {
    "locations": [{"path": "src/cache.py", "symbol": "Cache.get"}],
    "edit": {"tool": "str_replace", "artifact": "sha256:patch91..."},
    "tests": ["tests/test_cache.py::test_zero_ttl"]
  },
  "loss_mask": {"prompt": 0, "tool_observation": 0,
                "assistant_plan": 1, "assistant_action": 1},
  "quality": {"patch_applies": true, "gold_tests_pass": true}
}
```

D.3 统一轨迹事件模型

轨迹不应只存最终对话文本。建议以 append-only event 为事实源，每个事件至少包含 episode_id、单调递增 seq、event_type、producer、policy/harness/environment 版本、payload 或 artifact、token/cost 与时间。对 model action 额外保存 action-token span 和 rollout policy log-prob；对 tool result 保存 exit code、截断信息和原始 artifact hash；对 workspace mutation 保存 before/after tree hash。

表 14: 不同轨迹类型及默认训练用途

轨迹类型	必备证据	推荐用途	默认不做什么
直接成功	测试前后状态、最终 diff、主动提交	multi-turn SFT、成功策略分析	不把 tool observation 计入生成 loss
失败后恢复	错误动作、失败反馈、诊断与修正分支	recovery SFT、process preference、prefix branching	不删除失败前缀只保留最终答案
成败/优劣对	同题、同 harness、同预算的两条轨迹	DPO/排序器、成本与风险偏好	不跨任务或跨预算直接配对
在线 RL rollout	policy version、old log-prob、reward vector、stop reason	GRPO/GSPO/DAPO 类更新	不混入环境故障和未知 policy 轨迹
无效/危险负轨迹	loop、test tamper、越权证据与规则标签	policy/verifier、拒绝与安全评测	不作为正向 action imitation 目标

D.4 轨迹样例一：直接成功

该样本适合做 multi-turn SFT。训练时 observation 作为条件输入，通常只对 assistant 的短计划、tool call 与 final answer 计算 loss；若内部不训练显式长思维链，可把 PLAN 限制为可审计的目标/依据/下一步三元组。

Listing 16: 搜索–编辑–测试–提交的成功轨迹（JSONL）

```
{"episode_id":"ep_ok_17","seq":0,"event_type":"TASK",
 "payload":{"task_id":"swe_042","issue":"ttl=0 returns stale values"}}
{"episode_id":"ep_ok_17","seq":1,"event_type":"PLAN",
 "payload":{"goal":"locate ttl branch","next":"search Cache.get"}}
{"episode_id":"ep_ok_17","seq":2,"event_type":"TOOL_CALL",
 "payload":{"tool":"shell","cmd":"rg -n 'ttl' src tests"}}
{"episode_id":"ep_ok_17","seq":3,"event_type":"TOOL_RESULT",
 "payload":{"exit_code":0,"summary":"src/cache.py:81","artifact":"sha256:out11..."}}
{"episode_id":"ep_ok_17","seq":4,"event_type":"EDIT",
 "payload":{"path":"src/cache.py","patch":"sha256:patch12...","trees":["t01","t02"]}}
{"episode_id":"ep_ok_17","seq":5,"event_type":"TOOL_CALL",
 "payload":{"tool":"test","target":"test_zero_ttl"}}
{"episode_id":"ep_ok_17","seq":6,"event_type":"TOOL_RESULT","payload":{"passed":1,"failed":0}}
{"episode_id":"ep_ok_17","seq":7,"event_type":"FINISH","payload":{"reason":"solved"}}
```

D.5 轨迹样例二：失败后恢复

恢复轨迹比“干净成功轨迹”更能训练环境反馈利用：Agent 先做出局部合理但错误的修改，定向测试通过而回归测试失败，随后依据错误定位副作用并缩小 patch。应保留错误分支及其 observation；只保留最后三步会把样本错误地变成一次性成功。

Listing 17: 错误 patch 经回归反馈修正的恢复轨迹

```
{"episode_id":"ep_recover_08","seq":0,"event_type":"TASK",
 "payload":{"task_id":"swe_042"}}
{"episode_id":"ep_recover_08","seq":1,"event_type":"EDIT",
 "payload":{"patch":"sha256:broad_fix...","scope_files":2}}
{"episode_id":"ep_recover_08","seq":2,"event_type":"TEST_RESULT",
 "payload":{"suite":"targeted","passed":1,"failed":0}}
{"episode_id":"ep_recover_08","seq":3,"event_type":"TEST_RESULT",
 "payload":{"suite":"regression","passed":183,"failed":2,
            "error_class":"cache_disabled_semantics_changed"}}
{"episode_id":"ep_recover_08","seq":4,"event_type":"DIAGNOSIS",
 "payload":{"evidence":"ttl=None and ttl=0 were conflated",
            "decision":"revert store.py; narrow condition in cache.py"}}
{"episode_id":"ep_recover_08","seq":5,"event_type":"EDIT",
 "payload":{"patch":"sha256:narrow_fix...","scope_files":1}}
{"episode_id":"ep_recover_08","seq":6,"event_type":"TEST_RESULT",
 "payload":{"suite":"full","passed":185,"failed":0}}
{"episode_id":"ep_recover_08","seq":7,"event_type":"FINISH",
 "payload":{"reason":"solved_after_recovery","recovery_count":1}}
```

D.6 轨迹样例三：偏好对、RL rollout 与危险负例

偏好学习必须保证配对可比：相同 task、base commit、Harness、context 和预算。如下 chosen/rejected 都解决任务，但 chosen 修改更小、测试更全、成本更低，因此标签不是简单的 success > failure。

Listing 18: 同题轨迹偏好对

```
{
  "pair_id": "pref_042_03", "task_id": "swe_042",
  "control": {"harness":"h_9c2","max_turns":40,"context":65536},
  "chosen": {"episode_id":"ep_ok_17","resolved":true,
             "changed_files":1,"turns":8,"full_test":true},
  "rejected": {"episode_id":"ep_ok_19","resolved":true,
               "changed_files":4,"turns":31,"full_test":false},
  "preference": {"winner":"ep_ok_17",
                 "reasons":["smaller_diff","regression_tested","lower_cost"],
                 "label_source":"execution_plus_rules"}
}
```

在线 RL 记录必须能判断样本是否由当前/允许滞后的 policy 产生，以及失败来自 policy 还是基础设施。Reward 建议先存向量、后组合；否则改变权重后无法离线重算和诊断 reward hacking。

Listing 19: 可用于 group-relative RL 的 rollout 记录

```
{
  "rollout_id":"ro_g17_06", "group_id":"g17", "task_id":"swe_042",
  "policy":{"checkpoint":"ckpt_1200","version":1200,"staleness":1},
  "sampling":{"seed":7306,"temperature":1.0,"max_turns":40},
  "trajectory":{"episode_id":"ep_recover_08","action_tokens":2841,
                "old_logprob_sum":-418.73,"stop_reason":"DONE"},
  "reward":{"f2p":1.0,"p2p":1.0,"progress":0.2,"submit":0.1,
            "turn_cost":-0.08,"unsafe":0.0,"test_tamper":0.0,
            "total":2.22},
  "infrastructure":{"container":"OK","grader":"OK","flaky":false},
  "loss_mask":{"policy_update":1,"tool_observation_tokens":0}
}
```

危险负例应保存触发规则和证据位置，而非只有总 reward。下例中定向测试被直接修改，即使最终公开测试变绿也不能算成功；它适合训练 action policy、outcome verifier 和红队评测，但不应让模型模仿该 edit。

Listing 20: 测试投机负轨迹摘要

```
{
  "episode_id":"ep_bad_04","task_id":"swe_042","resolved_public":true,
  "violations":[
    {"type":"test_tamper","seq":9,"path":"tests/test_cache.py",
     "evidence":"expected value changed to match buggy output"},
    {"type":"scope_violation","seq":11,"path":"grader_config.json"}
  ],
  "terminal":{"reason":"POLICY_BLOCKED","reward":0.0},
  "training_use":{"positive_sft":false,"policy_negative":true,
                  "verifier_negative":true,"rl_update":true}
}
```

D.7 入库前的完整性约束

轨迹生产完成后至少执行四类约束：（1）**顺序完整性**：seq 连续，tool call 与 result 一一对应；（2）**状态完整性**：workspace mutation 前后 tree hash 可验证，final patch 能从事件重建；（3）**版本完整性**：task、policy、scaffold、harness、container、evaluator 均有不可变版本；（4）**标签完整性**：success、stop reason、reward components、infra mask 与 violation 可分别查询。训练集发布时还应给出按 task/repo 去重后的 episode 数、actual action tokens、成功率、恢复率、环境失败率和轨迹长度分布，而不只报告原始事件条数。

参考文献

[1] T. Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS, 2023.

[2] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.

[3] R. Nakano et al. *WebGPT: Browser-Assisted Question-Answering with Human Feedback*. arXiv:2112.09332, 2021.

[4] OpenAI. *Deep Research System Card*. System card, 2025.

[5] C. E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.

[6] J. Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS, 2024.

[7] X. Wang et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. arXiv:2407.16741, 2024.

[8] Z. Jiang, D. Lo, and Z. Liu. *Agentic Software Issue Resolution with Large Language Models: A Survey*. arXiv:2512.22256, 2025.

[9] I. Ceka et al. *Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study*. arXiv:2506.08311, 2025.

[10] H. Zhou et al. *Self-Evolving Coding Agents*. arXiv:2608.03392, 2026.

[11] C.-P. Hsieh et al. *RULER: What's the Real Context Size of Your Long-Context Language Models?* COLM, 2024.

[12] C. S. Xia et al. *Agentless: Demystifying LLM-based Software Engineering Agents*. arXiv:2407.01489, 2024.

[13] S. Huang et al. *OpenCoder: The Open Cookbook for Top-Tier Code Large Language Models*. ACL, 2025; arXiv:2411.04905.

[14] B. Hui et al. *Qwen2.5-Coder Technical Report*. arXiv:2409.12186, 2024.

[15] A. Lozhkov et al. *StarCoder 2 and The Stack v2: The Next Generation*. Technical report, 2024.

[16] D. Guo et al. *DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence*. arXiv:2401.14196, 2024.

[17] N. Muennighoff et al. *OctoPack: Instruction Tuning Code Large Language Models*. ICLR, 2024.

[18] C. K. Lo et al. *Pull Requests as a Training Signal for Repo-Level Code Editing*. arXiv:2602.07457, 2026.

[19] J. Pan et al. *Training Software Engineering Agents and Verifiers with SWE-Gym*. arXiv:2412.21139, 2024.

[20] J. Yang et al. *SWE-smith: Scaling Data for Software Engineering Agents*. arXiv:2504.21798, 2025.

[21] N. Jain et al. *R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents*. arXiv:2504.07164, 2025.

[22] I. Badertdinov et al. *SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents*. arXiv:2505.20411, 2025.

[23] Y. Ding et al. *CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion*. NeurIPS Datasets and Benchmarks, 2023.

[24] T. Liu, C. Xu, and J. McAuley. *RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems*. ICLR, 2024.

[25] B. Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models*. ICLR, 2024.

[26] Y. Ding et al. *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens*. ICML, 2024.

[27] Y. Bai et al. *LongAlign: A Recipe for Long Context Alignment of Large Language Models*. Findings of EMNLP, 2024.

[28] X. Wang et al. *Executable Code Actions Elicit Better LLM Agents*. ICML, 2024.

[29] A. Golubev et al. *Training Long-Context, Multi-Turn Software Engineering Agents with Reinforcement Learning*. arXiv:2508.03501, 2025.

[30] Y. Wei et al. *SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution*. arXiv:2502.18449, 2025.

[31] H. Song et al. *SWE-Master: Unleashing the Potential of Software Engineering Agents via Post-Training*. arXiv:2602.03411, 2026.

[32] J. Da et al. *Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards*. arXiv:2506.11425, 2025.

[33] L. Sutawika et al. *CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents*. arXiv:2603.17829, 2026.

[34] Y. Wei et al. *Toward Training Superintelligent Software Agents through Self-Play SWE-RL*. arXiv:2512.18552, 2025.

[35] SWE-agent Team. *mini-SWE-agent: A Minimal Software Engineering Agent Harness*. Official repository; accessed 2026-08-07.

[36] SWE-bench Team. *SWE-bench Evaluation Harness Reference*. Official documentation; accessed 2026-08-07.

[37] Anthropic. *Building Effective AI Agents*. Engineering article, 2024.

[38] Anthropic. *Claude Code: Best Practices for Agentic Coding*. Engineering article, 2025.

[39] Anthropic. *Enabling Claude Code to Work More Autonomously*. Product and SDK announcement, 2025.

[40] Anthropic. *How Claude Remembers Your Project*. Claude Code documentation; accessed 2026-08-07.

[41] Anthropic. *Automate Workflows with Hooks*. Claude Code documentation; accessed 2026-08-07.

[42] Anthropic. *Create Custom Subagents*. Claude Code documentation; accessed 2026-08-07.

[43] Anthropic. *Sandboxing*. Claude Code documentation; accessed 2026-08-07.

[44] Anthropic. *Effective Harnesses for Long-Running Agents*. Engineering article, 2025.

[45] P. Rajasekaran. *Harness Design for Long-Running Application Development*. Anthropic Engineering, 2026.

[46] Anthropic. *How We Built Claude Code Auto Mode: A Safer Way to Skip Permissions*. Engineering article, 2026.

[47] M. Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2021.

[48] N. Jain et al. *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*. arXiv:2403.07974v2, 2024.

[49] T. Y. Zhuo et al. *BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions*. arXiv:2406.15877, 2024.

[50] Y. Lai et al. *DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation*. ICML, 2023.

[51] Microsoft Research. *SWE-bench-Live: Continuously Updated Multi-Language and Multi-OS Software Engineering Tasks*. Official repository; accessed 2026-08-07.

[52] M. Bavarian et al. *Efficient Training of Language Models to Fill in the Middle*. arXiv:2207.14255, 2022.

[53] SWE-agent Team. *Trajectories: Output Format and Replayable Agent History*. Official documentation; accessed 2026-08-07.

[54] Harbor Framework. *Agent Trajectory Interchange Format (ATIF)*. RFC 0001; accessed 2026-08-07.